

INT3401E: Artificial Intelligence

Lecture 02: Solving Problems by Searching Uniformed/Blind Search Strategies

Dr. Le Duc Trong
Faculty of IT, VNU-UET

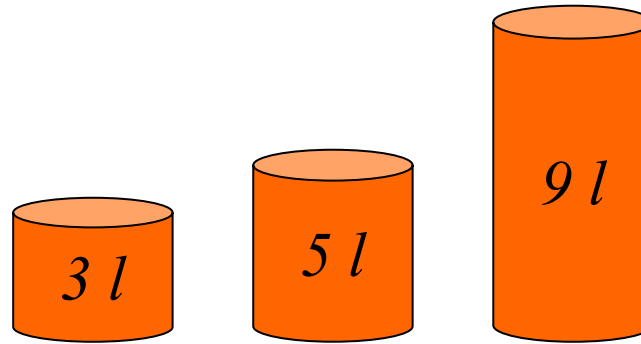


Outline: Problem solving by searching



- **Introduction to Problem Solving**
- **Uninformed search (Blind Search)**
 - Problem formulation
 - Search strategies: depth-first, breadth-first, uniform-cost search
- **Informed search (next lecturer)**
 - Search strategies: best-first, A*
 - Heuristic functions

Example: Measuring problem!

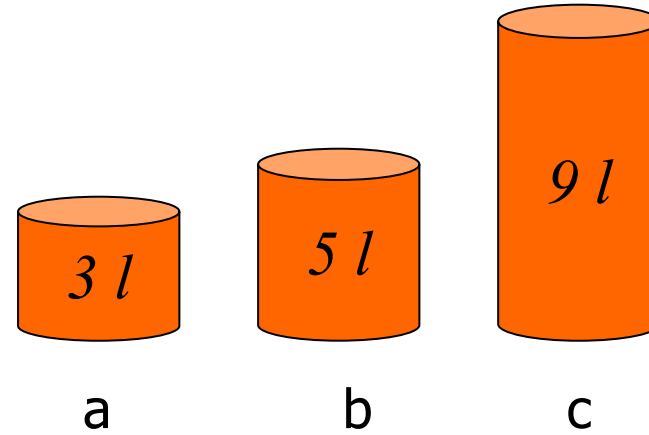


Problem: Using these three buckets,
measure 7 liters of water.

Example: Measuring problem!

- **(one possible) Solution:**

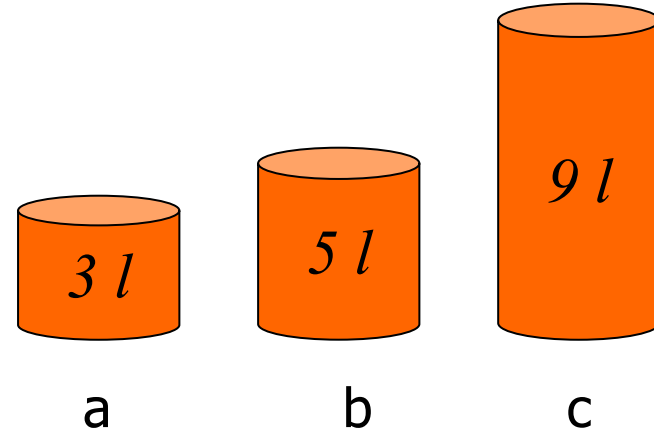
a	b	c	
0	0	0	start



Example: Measuring problem!

- **(one possible) Solution:**

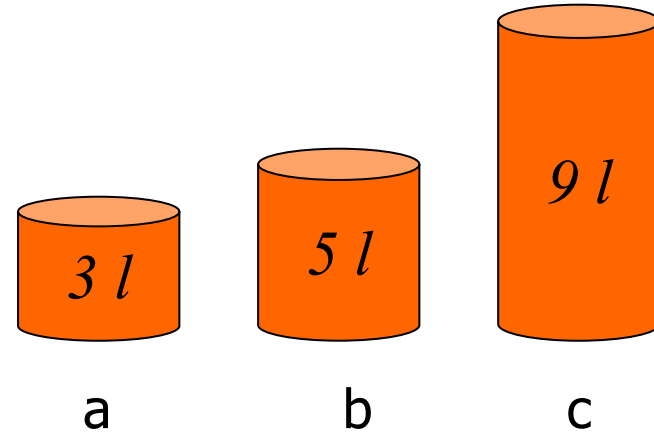
a	b	c	
0	0	0	start
3	0	0	



Example: Measuring problem!

- (one possible) Solution:

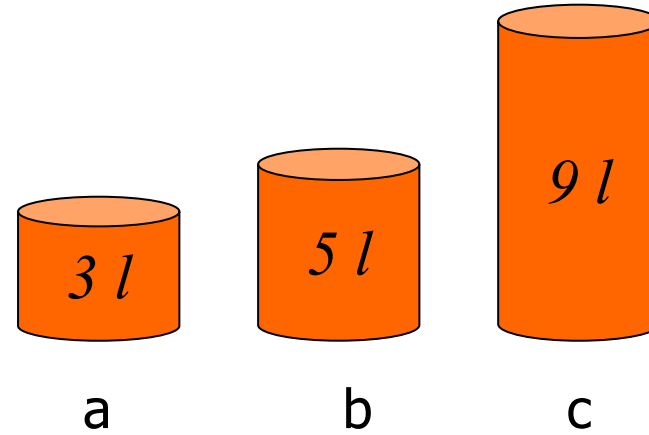
a	b	c	
0	0	0	start
3	0	0	
0	0	3	



Example: Measuring problem!

- (one possible) Solution:

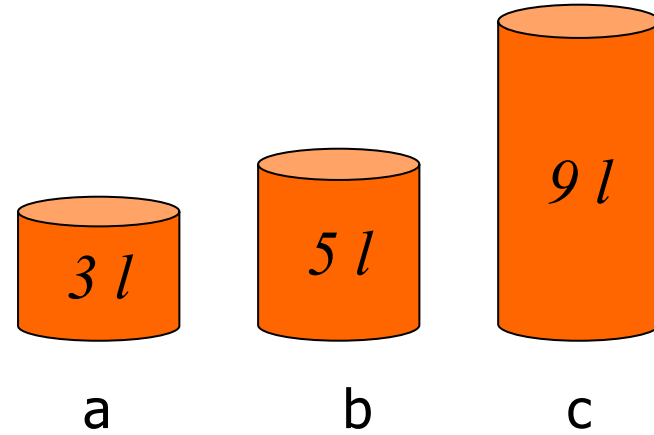
a	b	c	
0	0	0	start
3	0	0	
0	0	3	
3	0	3	



Example: Measuring problem!

- (one possible) Solution:

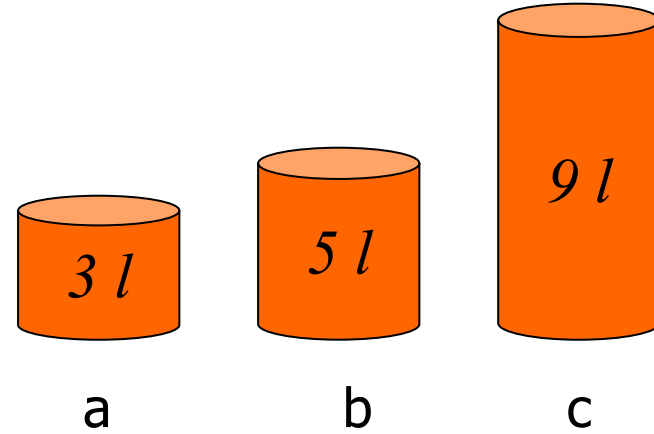
a	b	c	
0	0	0	start
3	0	0	
0	0	3	
3	0	3	
0	0	6	



Example: Measuring problem!

- (one possible) Solution:

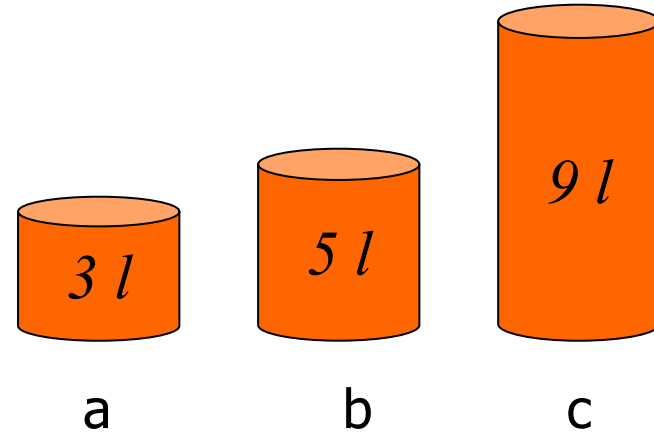
a	b	c	
0	0	0	start
3	0	0	
0	0	3	
3	0	3	
0	0	6	
3	0	6	



Example: Measuring problem!

- (one possible) Solution:

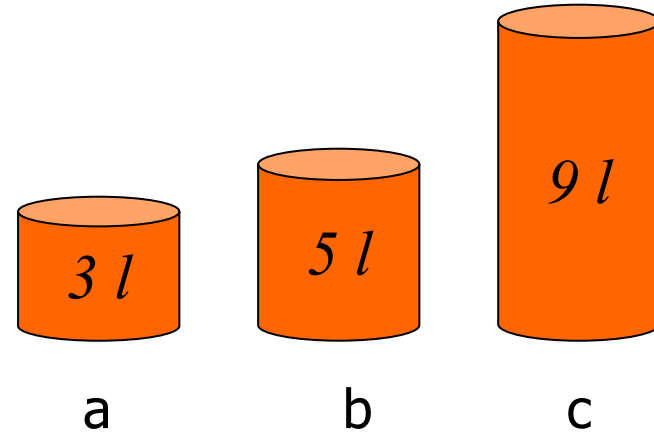
a	b	c	
0	0	0	start
3	0	0	
0	0	3	
3	0	3	
0	0	6	
3	0	6	
0	3	6	



Example: Measuring problem!

- (one possible) Solution:

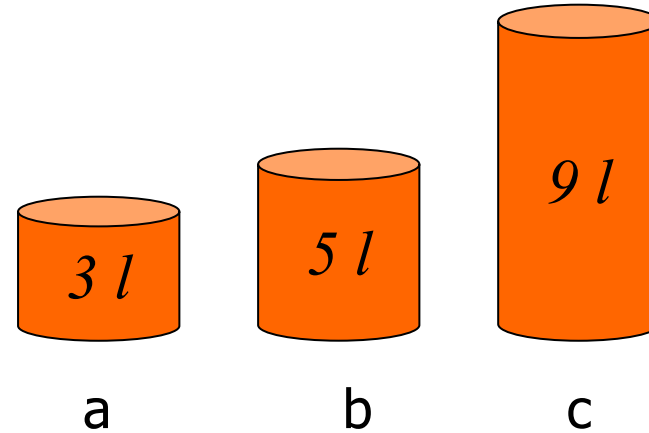
a	b	c	
0	0	0	start
3	0	0	
0	0	3	
3	0	3	
0	0	6	
3	0	6	
0	3	6	
3	3	6	



Example: Measuring problem!

- (one possible) Solution:

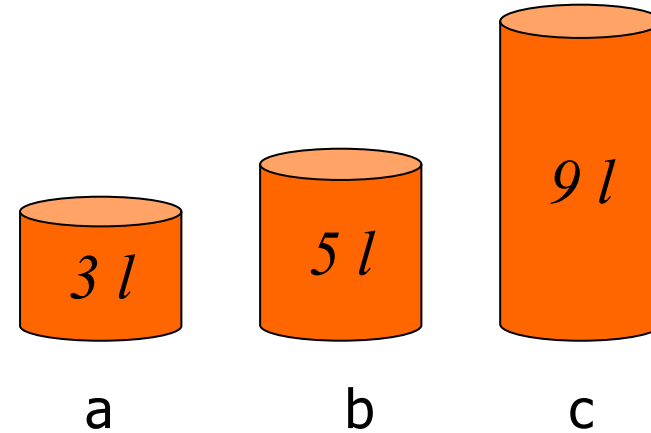
a	b	c	
0	0	0	start
3	0	0	
0	0	3	
3	0	3	
0	0	6	
3	0	6	
0	3	6	
3	3	6	
1	5	6	



Example: Measuring problem!

- (one possible) Solution:

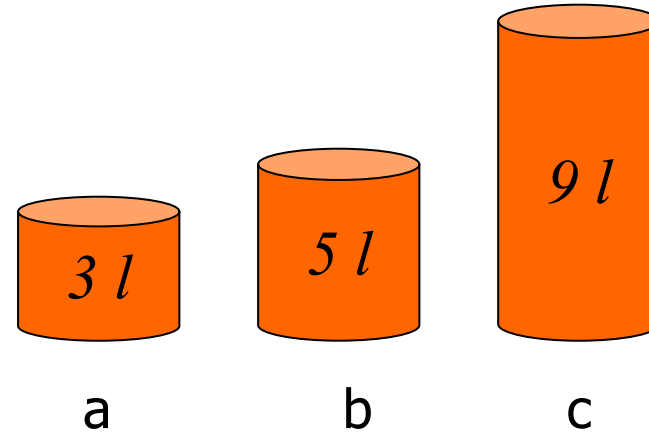
a	b	c	
0	0	0	start
3	0	0	
0	0	3	
3	0	3	
0	0	6	
3	0	6	
0	3	6	
3	3	6	
1	5	6	
0	5	7	goal



Example: Measuring problem!

- **Another Solution:**

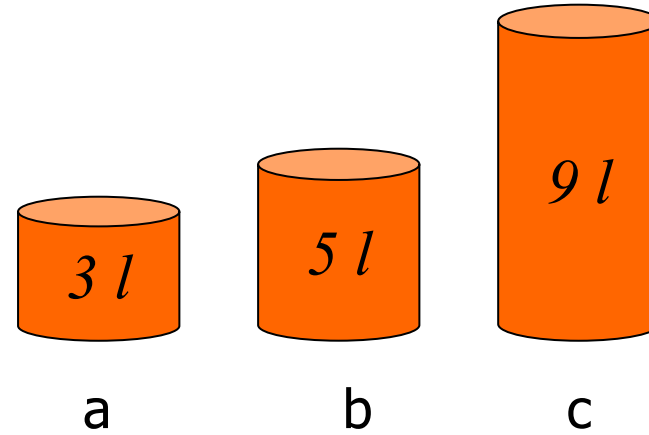
a	b	c	
0	0	0	start
0	5	0	



Example: Measuring problem!

- **Another Solution:**

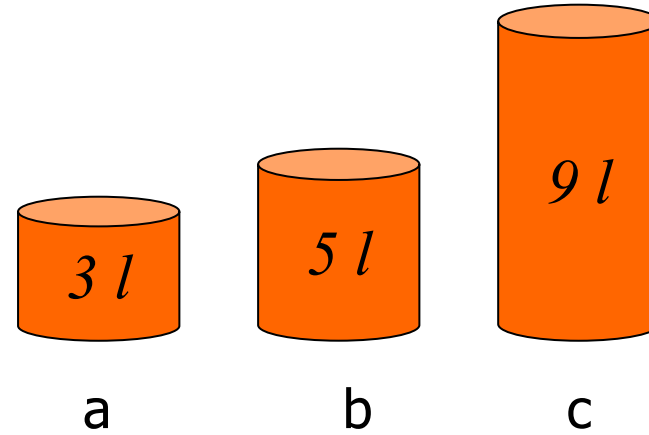
a	b	c	
0	0	0	start
0	5	0	
3	2	0	



Example: Measuring problem!

- **Another Solution:**

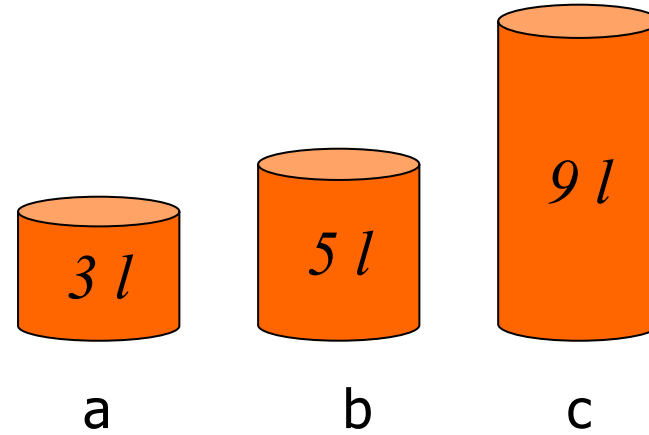
a	b	c	
0	0	0	start
0	5	0	
3	2	0	
3	0	2	



Example: Measuring problem!

- **Another Solution:**

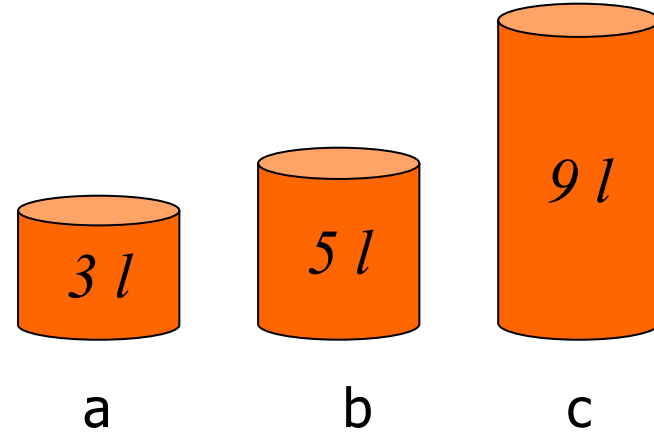
a	b	c	
0	0	0	start
0	5	0	
3	2	0	
3	0	2	
3	5	2	



Example: Measuring problem!

- **Another Solution:**

a	b	c	
0	0	0	start
0	5	0	
3	2	0	
3	0	2	
3	5	2	
3	0	7	goal



Which solution do we prefer?

- Solution 1:**

a	b	c	
0	0	0	start
3	0	0	
0	0	3	
3	0	3	
0	0	6	
3	0	6	
0	3	6	
3	3	6	
1	5	6	
0	5	7	goal

- Solution 2:**

a	b	c	
0	0	0	start
0	5	0	
3	2	0	
3	0	2	
3	5	2	
3	0	7	goal

Other Example:

A **farmer** wants to get his **cabbage**, **goat**, and **wolf** across a river. He has a boat that only holds two. He cannot leave the cabbage and goat alone or the goat and wolf alone. How many river crossings does he need?

4

5

6

7

no solution

Problem-Solving



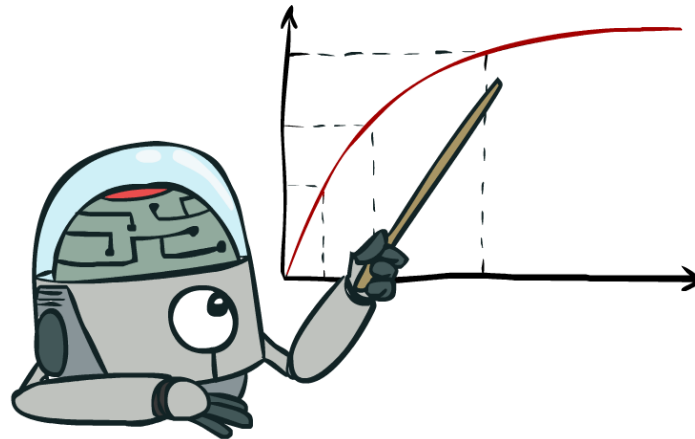
- **Problem solving:**
 - Goal formulation
 - Problem formulation (states, operators)
 - Search for solution
- **Problem formulation:**
 - Initial (start) state
 - Actions
 - Goal test
 - Path cost

Rational Decisions

- We'll use the term **rational** in a very specific, technical way:
 - Rational: *maximally achieving pre-defined goals*
 - Goals are expressed in terms of the **utility** of outcomes
 - World is uncertain, so we'll use **expected** utility

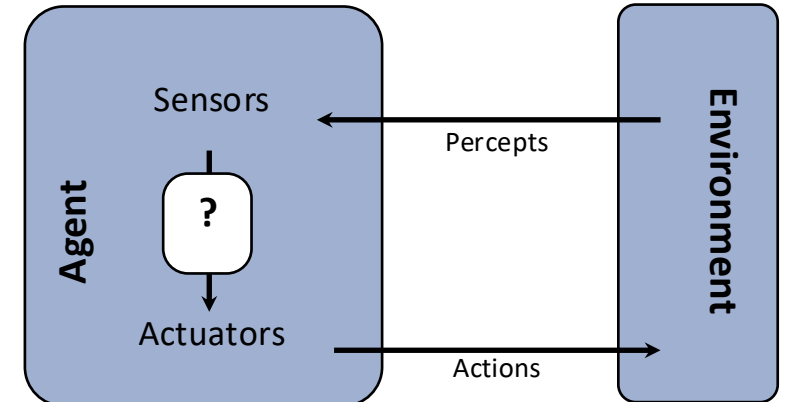
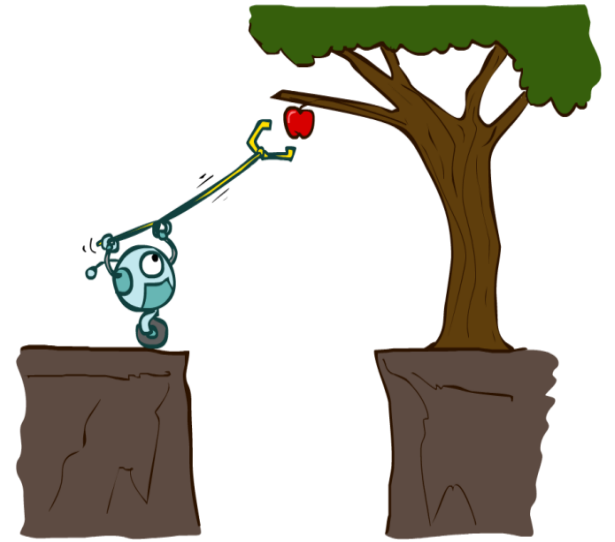
$$E[U(a)] = \sum_{s'} P(s'|s, a)U(s')$$

- Being **rational** means acting to **maximize your expected utility**



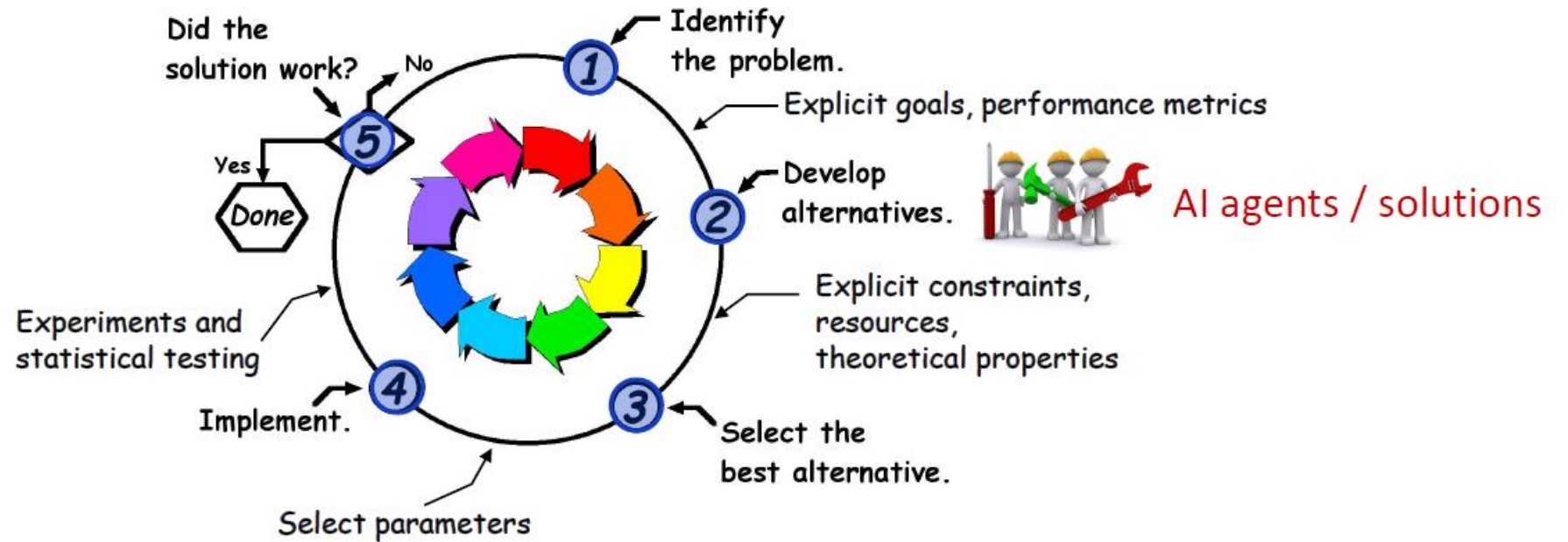
Design of Rational Agents

- An **agent** is an entity that perceives and acts.
- A **rational agent** selects actions that maximize its (expected) **utility**.
- Characteristics of the **percepts**, **environment**, and **action space** dictate techniques for selecting rational actions
- This course:
 - General AI techniques for a variety of problem types
 - Learning to recognize when and how a new problem can be solved with an existing technique



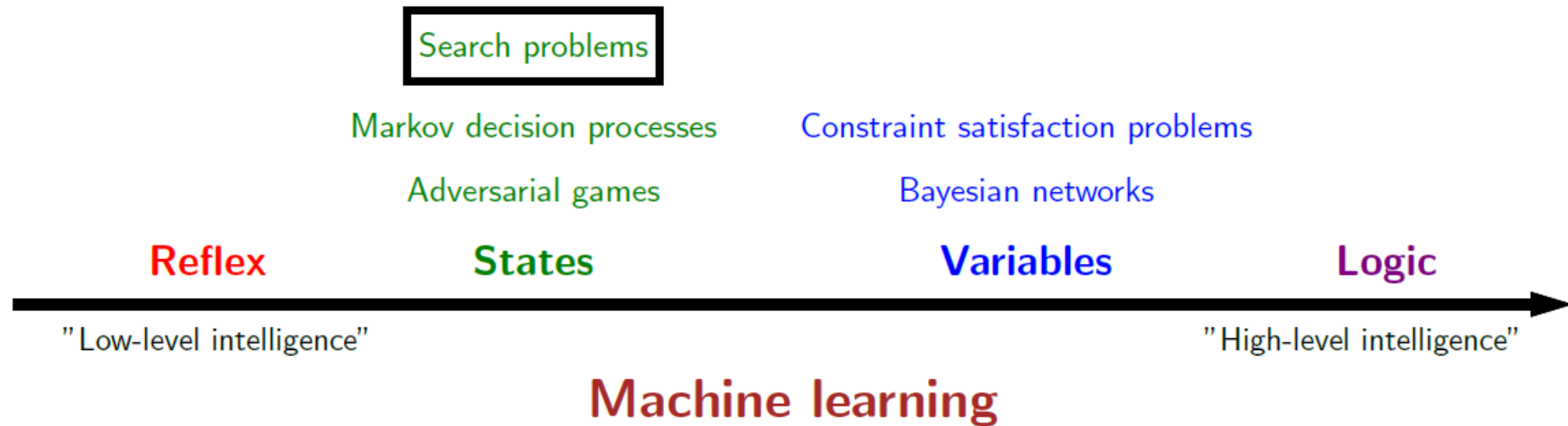
Problem-solving: (Iteratively) Selecting the right agent design

Steps to problem-solving



- The design of an **AI agent** for a complex task might require the integration of multiple components and techniques to deal with different aspects of the problem

Search Problems



Learning to Reason with LLMs (12/9/24): <https://openai.com/index/learning-to-reason-with-llms/>

Search problems (Deterministic Planning)

Task environment:

- ✓ Fully observable
- ✓ Known
- ✓ Deterministic
- ✓ Discrete
- ✓ Sequential
- ✓ Single agent

Objective:

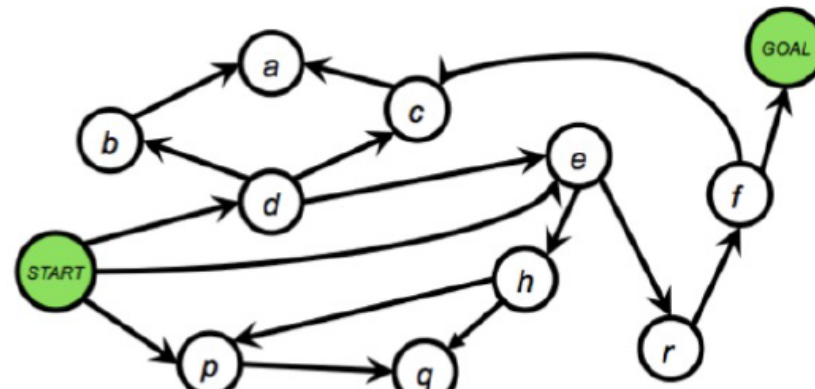
The agent has well defined **goals** that aims to reach at the minimum cost, starting from a defined start state

Output: A **plan** that takes from the *start state* to a *goal state* at the *minimum cost*

Plan: A *sequence of decisions* to be executed from start state in *open-loop*

Problem representation:

A *graph*, where nodes are *problem states* and arcs are *feasible actions* (state transitions)

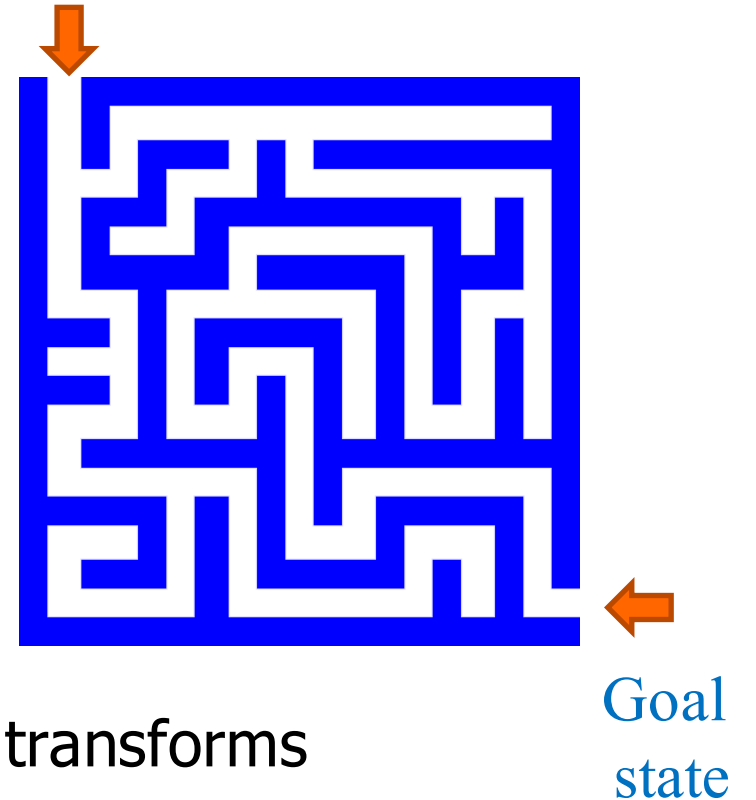


Activate Winc
Go to Settings to a

Search problem components

- **Start state**
- **Actions:** available to the agent in each state
- **Transition model**
 - the state resulting from doing action a in state s
- **Goal state**
- **Path cost**
 - Assume that it is a sum of nonnegative *step costs* ($c(s, a, s') \geq 0$)
- A **solution** is a sequence of actions (a plan) which transforms the **start state** to a **goal state**
- The **optimal solution** is the sequence of actions that gives the *lowest* path cost for reaching the goal

Start
state



Example: Romania

- On vacation in Romania; currently in Arad
- Flight leaves tomorrow from Bucharest

- **Start state**

- Arad

- **Actions**

- Go from one city to another

- **Transition model**

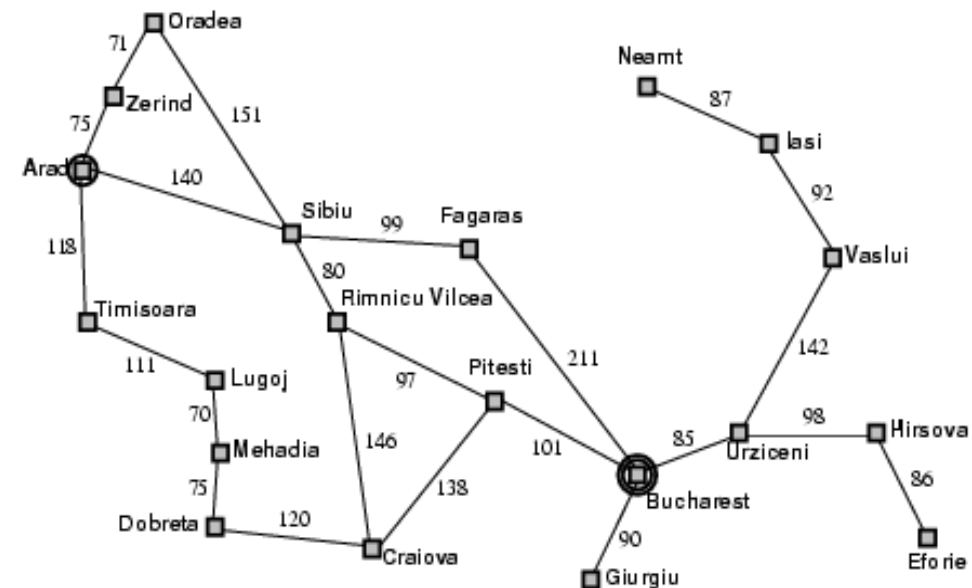
- If you go from city A to city B, you end up in city B

- **Goal state**

- Bucharest

- **Path cost**

- Sum of edge costs (total distance traveled)



Task Environment - PEAS

Performance measure

- -1 per step; +10 food; +500 win; -500 die;
+200 hit scared ghost

Environment

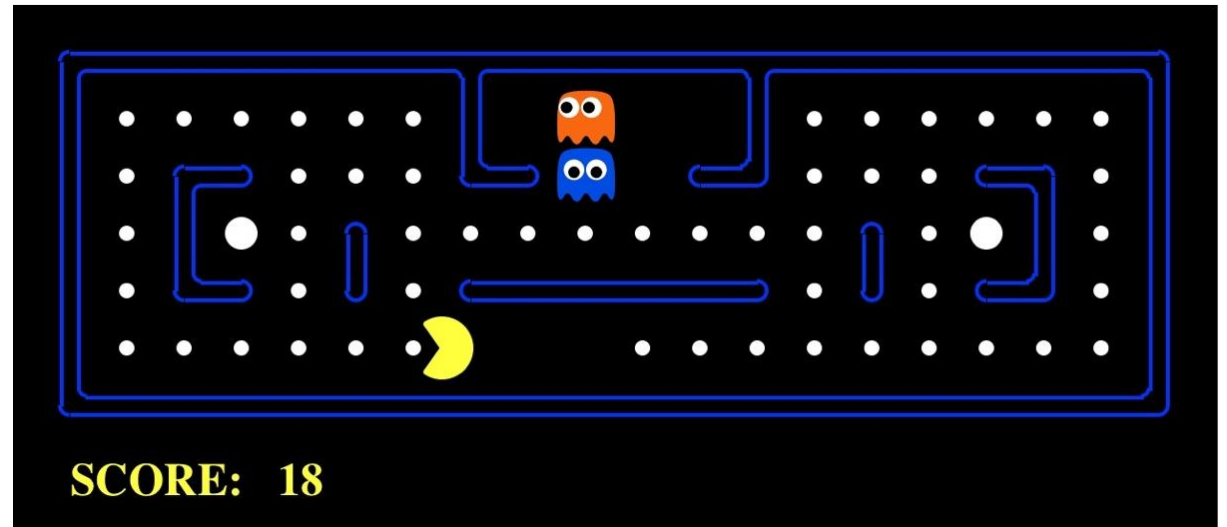
- Pacman dynamics (incl ghost behavior)

Actuators

- North, South, East, West, (Stop)

Sensors

- Entire state is visible



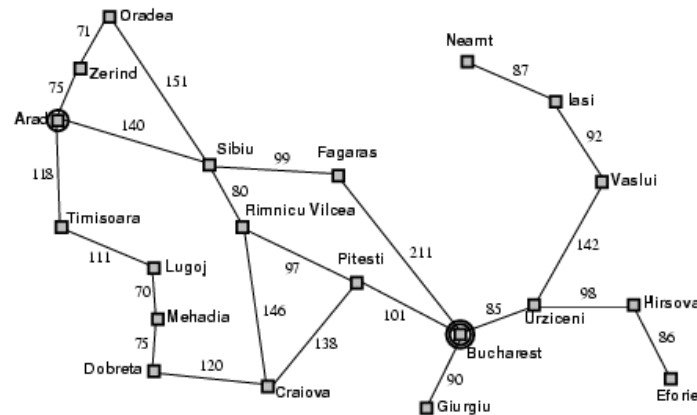
Environment Types – Quiz?



	Pacman	Taxi
Fully or partially observable		
Single agent or multi-agent		
Deterministic or stochastic		
Static or dynamic		
Discrete or continuous		

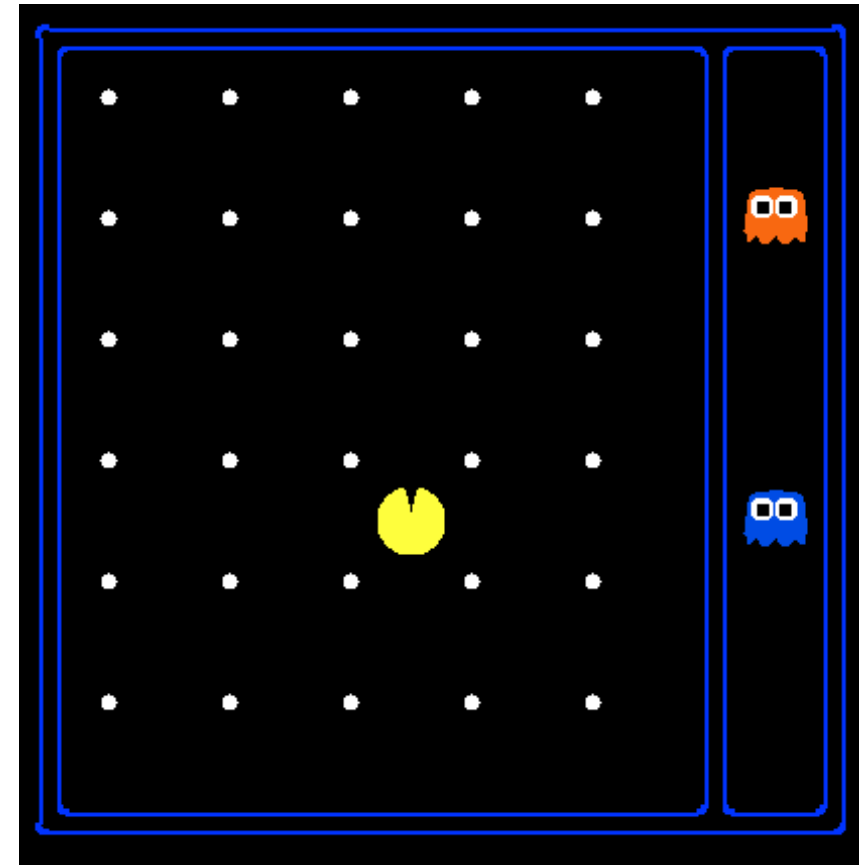
What's in a State Space?

- The start state, actions, and transition model define the **state space** of the problem
 - The set of all states reachable from start state by any sequence of actions
 - Determined by **start state + actions + transition model**
 - Can be represented as a **directed graph** where the nodes are states and links between nodes are actions
- What is the state space for the Romania problem?



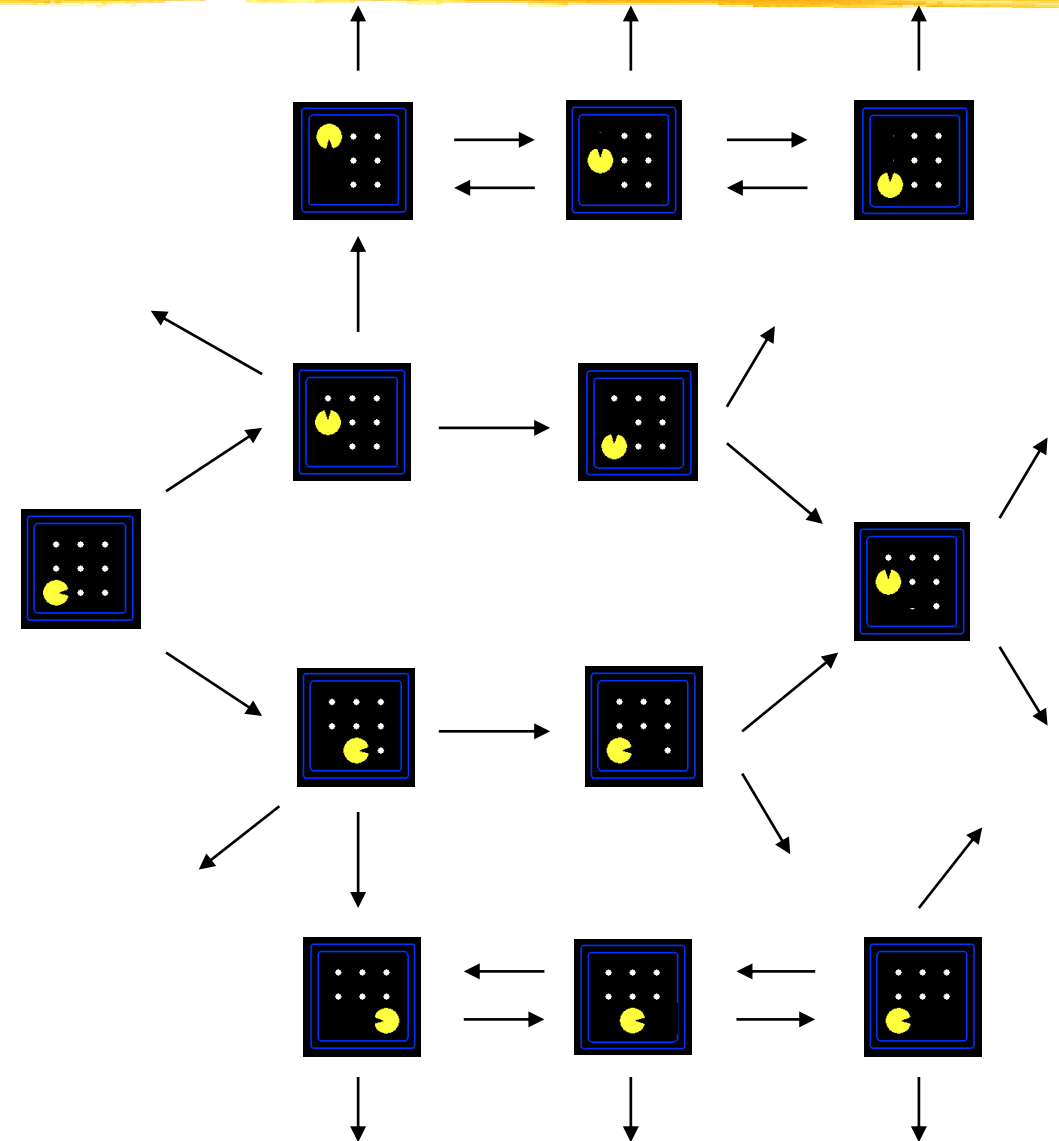
State Space Sizes?

- **World state:**
 - Agent positions: 120
 - Food count: 30
 - Ghost positions: 12
 - Agent facing: NSEW
- **How many**
 - World states?
 $120 \times (2^{30}) \times (12^2) \times 4$
 - States for pathing?
120
 - States for eat-all-dots?
 $120 \times (2^{30})$

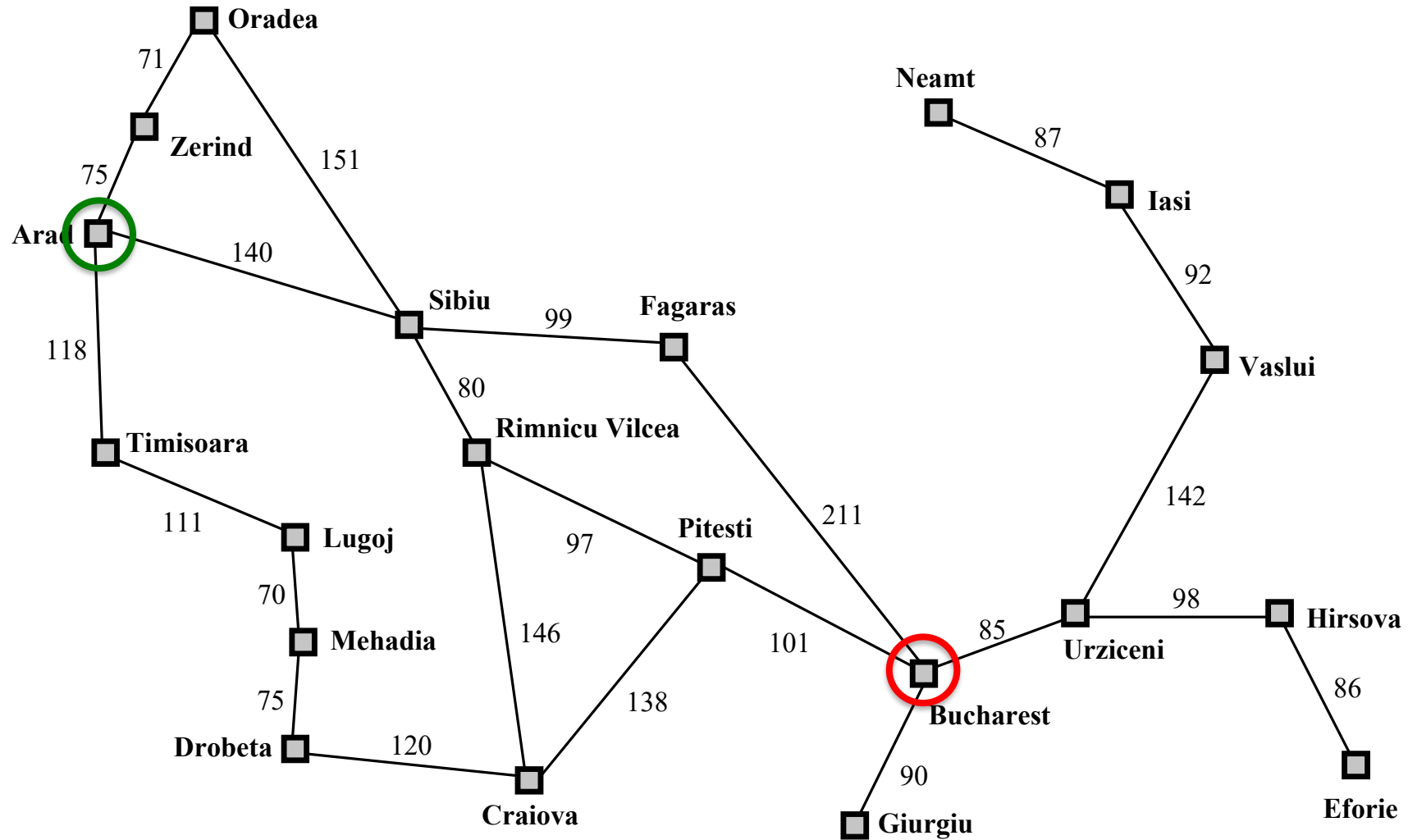


State Space Graphs

- **State space graph:** A mathematical representation of a search problem
 - Nodes are (abstracted) world configurations
 - Arcs represent transitions resulting from actions
 - The goal test is a set of goal nodes (maybe only one)
- In a state space graph, **each state occurs only once!**
- We can rarely build this full graph in memory (it's too big), but it's a useful idea



More Examples



Example: 8-puzzle

5	4	
6	1	8
7	3	2

start state

1	2	3
8		4
7	6	5

goal state

- State (trạng thái):
- Actions :
- Goal test:
- Path cost:

Example: 8-puzzle

5	4	
6	1	8
7	3	2

start state

1	2	3
8		4
7	6	5

goal state

- State: integer location of tiles (ignore intermediate locations)
- Actions: moving blank left, right, up, down (ignore jamming)
- Goal test: does state match goal state?
- Path cost: 1 per move

Example: 8-puzzle

5	4	
6	1	8
7	3	2

start state

1	2	3
8		4
7	6	5

goal state

Why search algorithms?

- 8-puzzle has 362,880 states
- 15-puzzle has 10^{12} states
- 24-puzzle has 10^{25} states

So, we need a principled way to look for a solution in these huge search spaces...

Search



- **Given:**
 - **Start state**
 - **Actions**
 - **Transition model**
 - **Goal state**
 - **Path cost**
- **How do we find the optimal solution?**
 - How about building the state space and then using Dijkstra's shortest path algorithm?
 - Complexity of Dijkstra's is $O(E + V \log V)$, where V is the size of the state space
 - The state space may be huge!

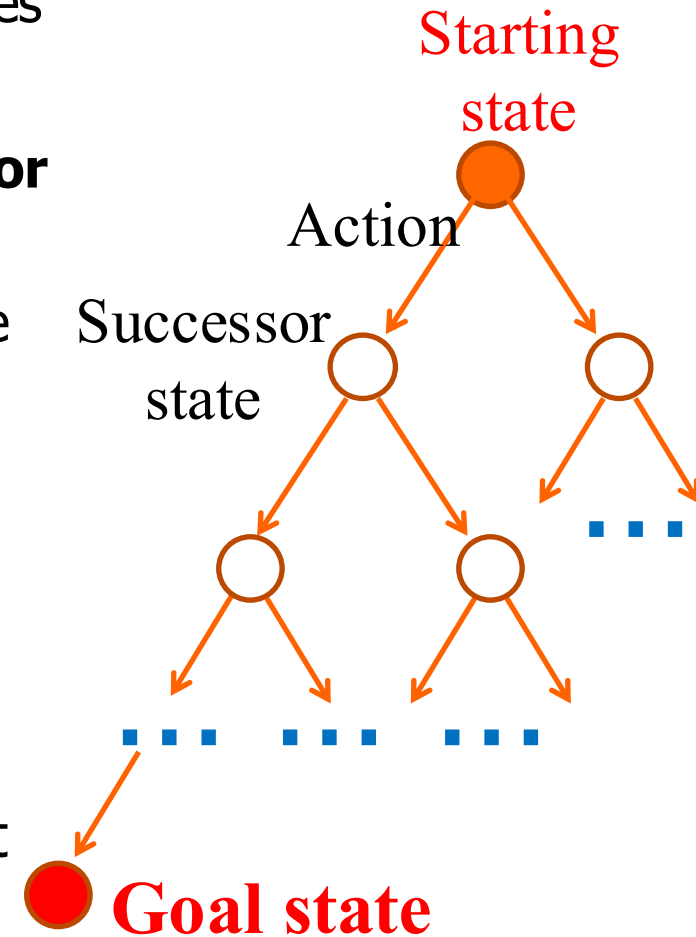
Search: Basic idea



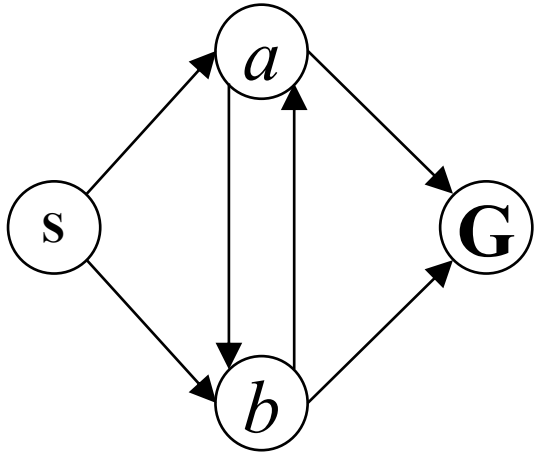
- Let's begin at the start state and **expand** it by making a list of all possible successor states
- Maintain a **frontier** or a list of unexpanded states
- At each step, pick a state from the frontier to expand
- Keep going until you reach a goal state
- Try to expand as few states as possible

Search tree

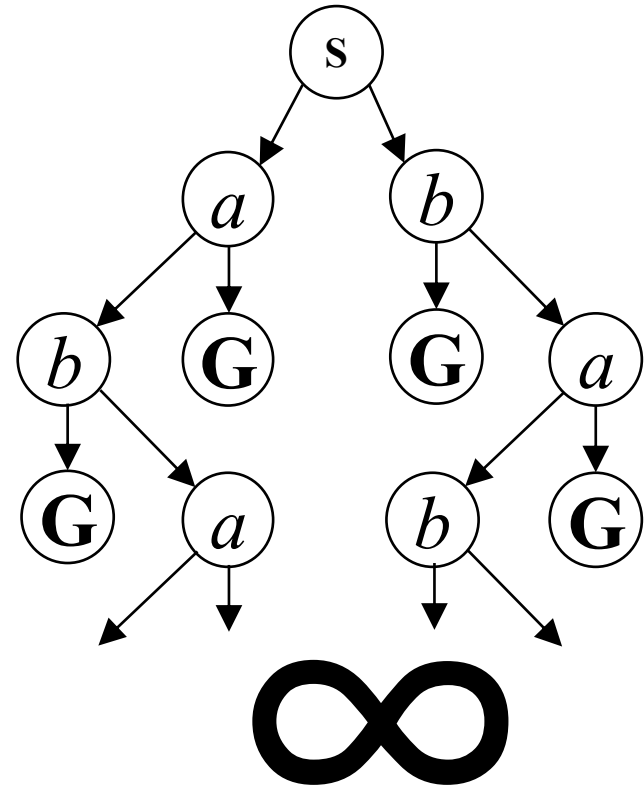
- “What if” tree of sequences of actions and outcomes
- The root node corresponds to the starting state
- The children of a node correspond to the **successor states** of that node’s state
- A path through the tree corresponds to a sequence of actions
 - A solution is a path ending in the goal state
- **Nodes vs. states**
 - A state is a representation of the world, while a node is a data structure that is part of the search tree
 - Node has to keep pointer to parent, path cost, possibly other info



Consider this 4-state graph:



How big is its search tree (from S)?



Important: Lots of repeated structure in the search tree!

Tree Search Algorithm Outline

```
Function General-Search(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy search
    if the node contains a goal state then
      return the corresponding solution
    else expand the node and add resulting nodes to the search tree
  end
```

Finding a solution

Solution: is ???

```
Function General-Search(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add resulting nodes to the search tree
  end
```

Finding a solution

Solution: is a sequence of actions that bring you from current state to the goal state.

```
Function General-Search(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add resulting nodes to the search tree
  end
```

Strategy: The search strategy is determined by ???

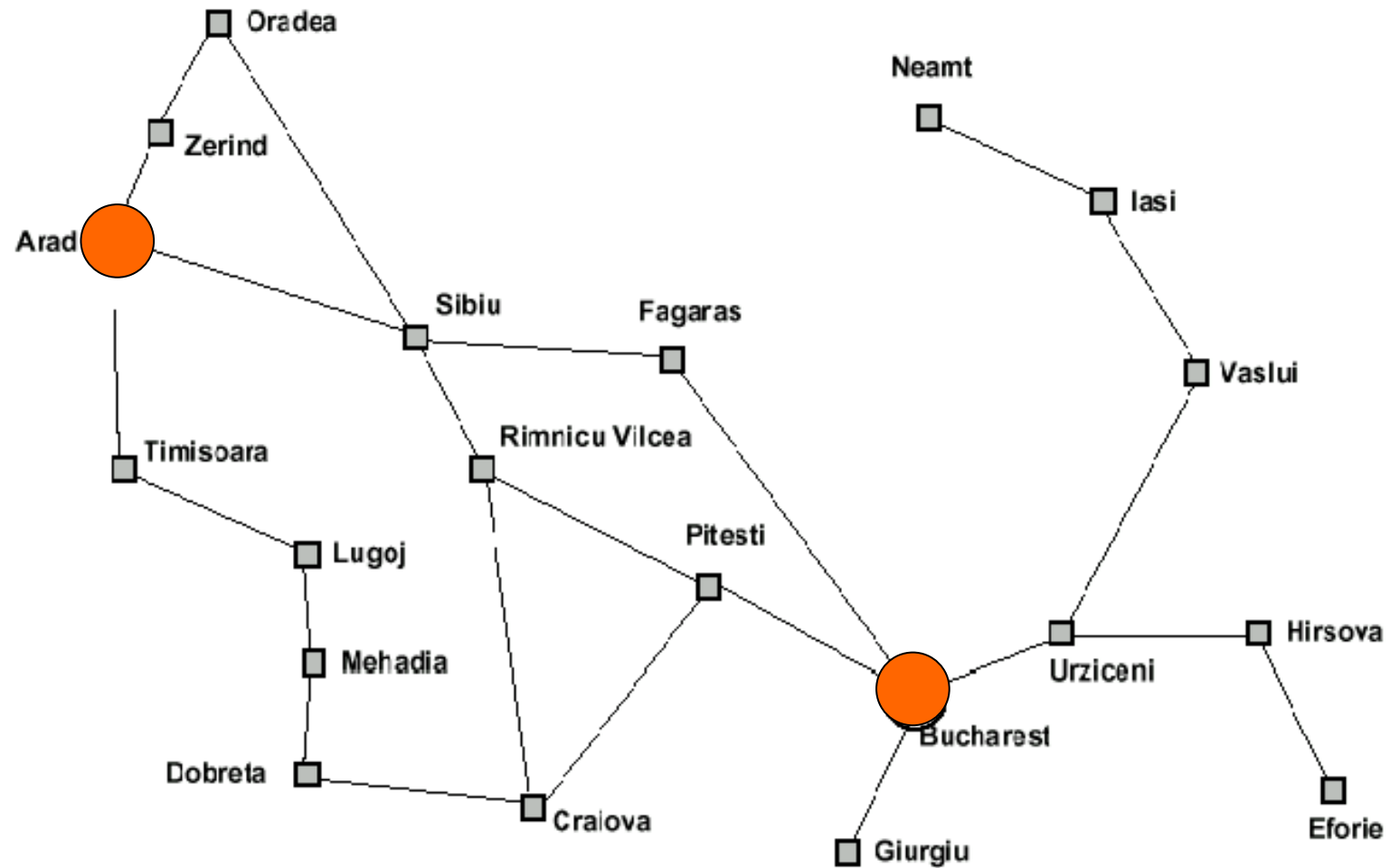
Finding a solution

Solution: is a sequence of actions that bring you from current state to the goal state

```
Function General-Search(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add resulting nodes to the search tree
  end
```

Strategy: The search strategy is determined by the **order** in which the nodes are expanded.

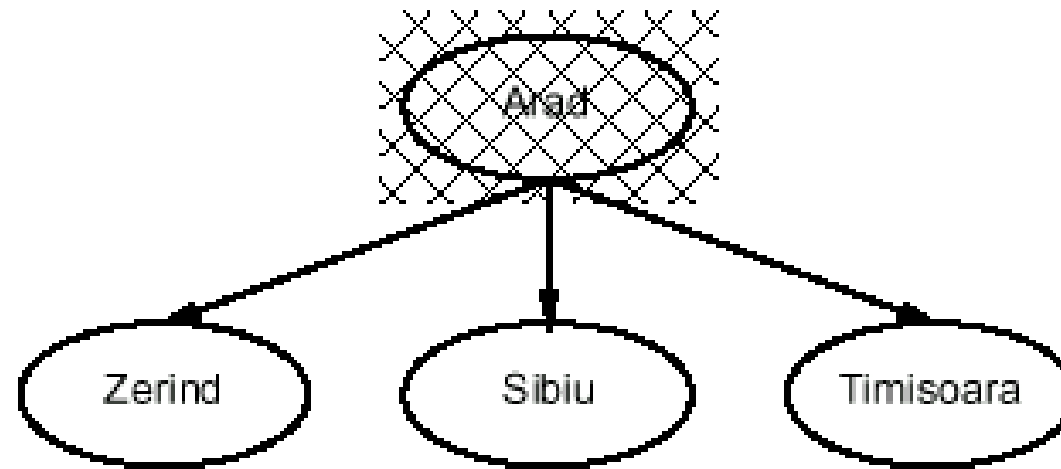
Example: Traveling from Arad To Bucharest



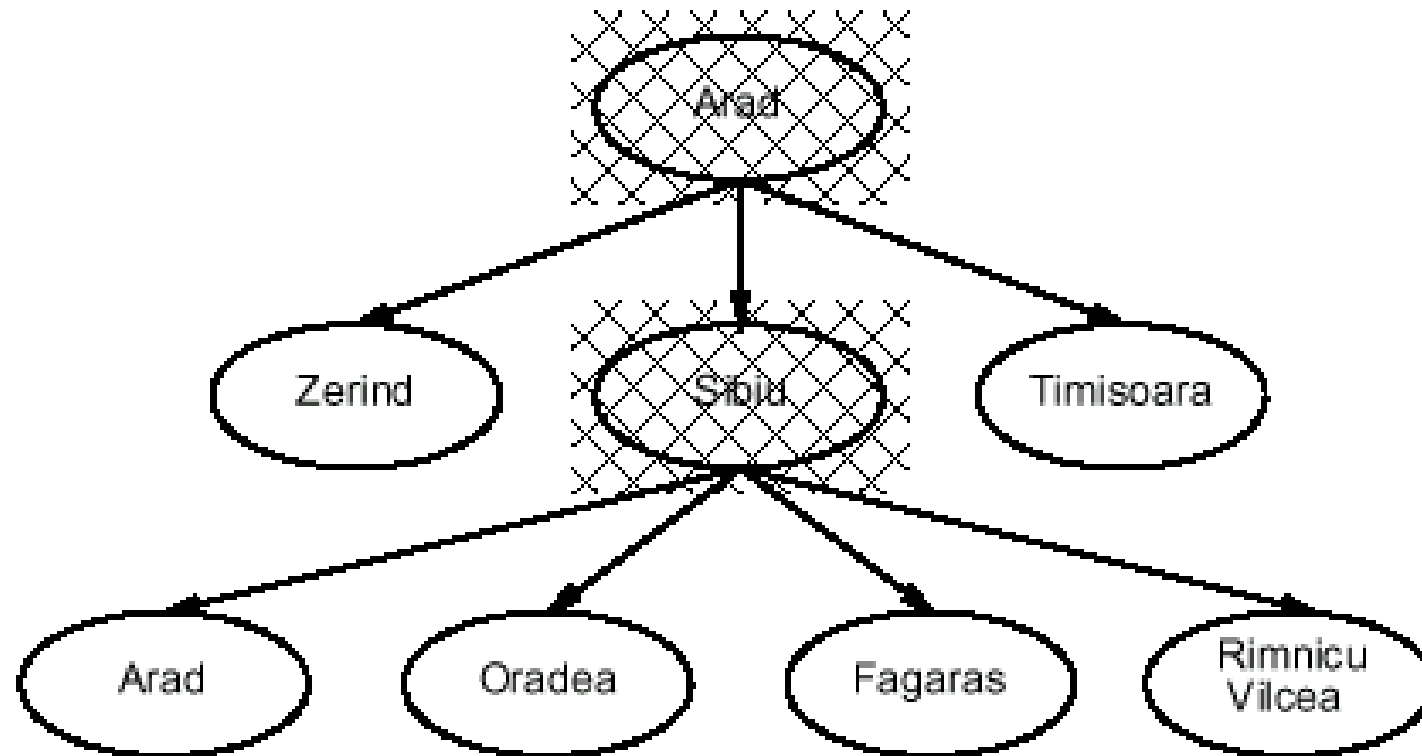
Tree search example



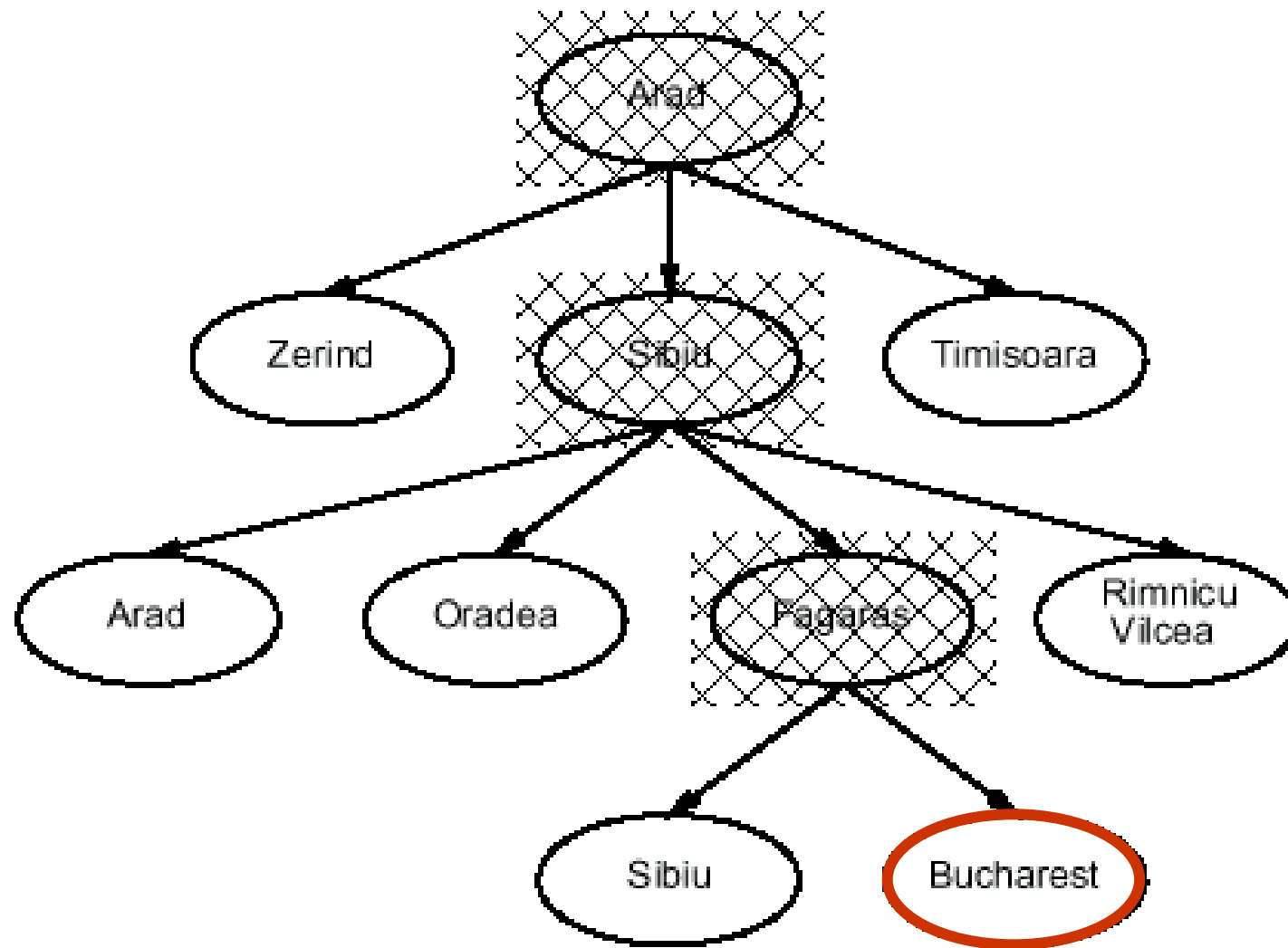
Tree search example



Tree search example



Tree search example



Implementation of search algorithms

```
Function General-Search(problem, Queuing-Fn) returns a solution, or failure
  nodes ← make-queue(make-node(initial-state[problem]))
  loop do
    if nodes is empty then return failure
    node ← Remove-Front(nodes)
    if Goal-Test[problem] applied to State(node) succeeds then return node
    nodes ← Queuing-Fn(nodes, Expand(node, Operators[problem]))
  end
```

Queuing-Fn(*queue*, *elements*) is a queuing function that inserts a set of elements into the queue and determines the order of node expansion. Varieties of the queuing function produce varieties of the search algorithm.

Function TREE_SEARCH(problem) returns a solution, or failure

initialize the frontier as a specific work list (stack, queue, priority queue)

add initial state of problem to frontier

loop do

if the frontier is empty then

return failure

choose a node and remove it from the frontier

if the node contains a goal state then

return the corresponding solution

for each resulting child from node

add child to the frontier

Function GRAPH_SEARCH(problem) returns a solution, or failure

initialize the explored set to be empty

initialize the frontier as a specific work list (stack, queue, priority queue)

add initial state of problem to frontier

loop do

if the frontier is empty then

return failure

choose a node and remove it from the frontier

if the node contains a goal state then

return the corresponding solution

add the node state to the explored set

for each resulting child from node

if the child state is not already in the frontier or explored set then

add child to the frontier

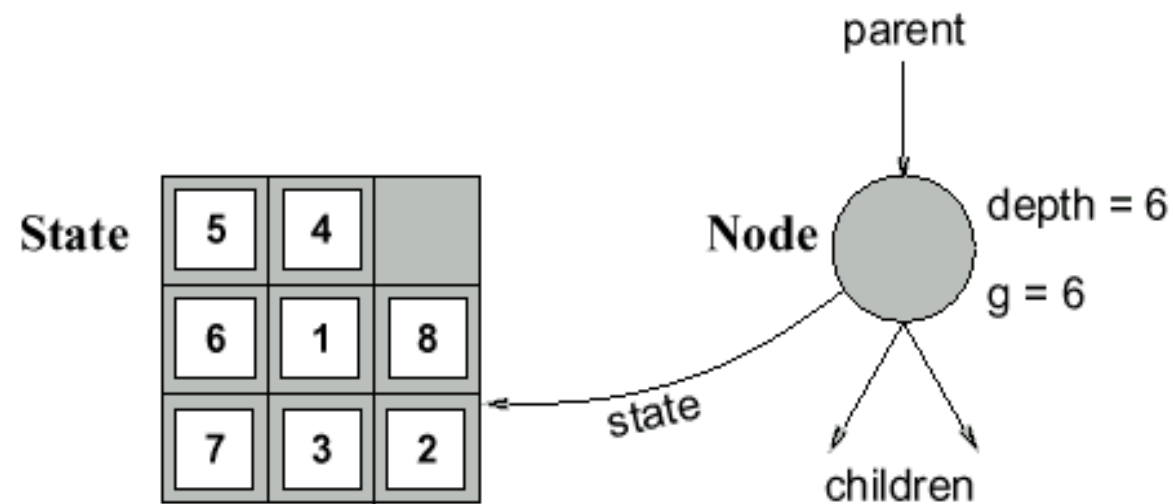
Encapsulating *state* information in *nodes*

A *state* is a (representation of) a physical configuration

A *node* is a data structure constituting part of a search tree

includes *parent*, *children*, *depth*, *path cost* $g(x)$

States do not have parents, children, depth, or path cost!



The EXPAND function creates new nodes, filling in the various fields and using the OPERATORS (or SUCCESSORFN) of the problem to create the corresponding states.

Evaluation of search strategies

- A search strategy is defined by **picking the order of node expansion**
- How do we will **score the performance** of different approaches for finding a solution?



Completeness

Guaranteed to find a solution when there is one?



Optimality

Finds the best solution?



Time

How long does it take to find a solution?



Space

How much memory needed to perform the search?

- **Parameters for space and time complexity:**
 - b : maximum branching factor of the search tree
 - d : depth of the least-cost solution
 - m : maximum depth of the state space (may be ∞)

Complexity



- Why worry about complexity of algorithms?
 - because a problem may be solvable in principle but may take too long to solve in practice

Complexity: Tower of Hanoi

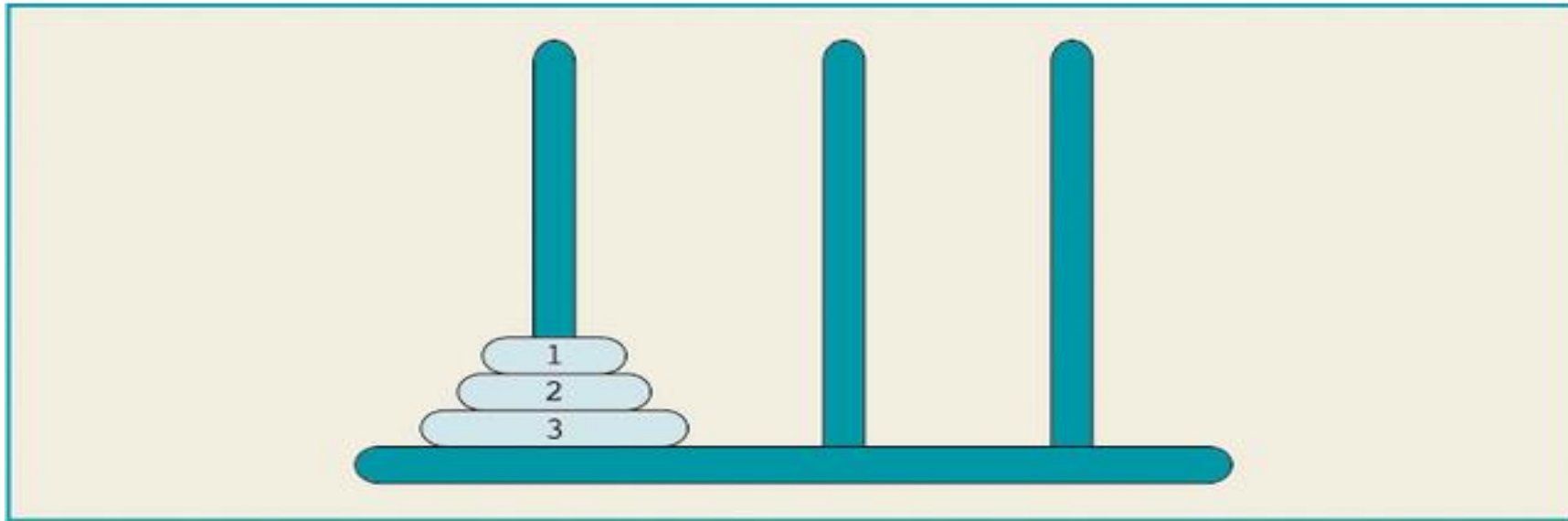


Figure 11-6 Tower of Hanoi problem with three disks

Complexity: Tower of Hanoi

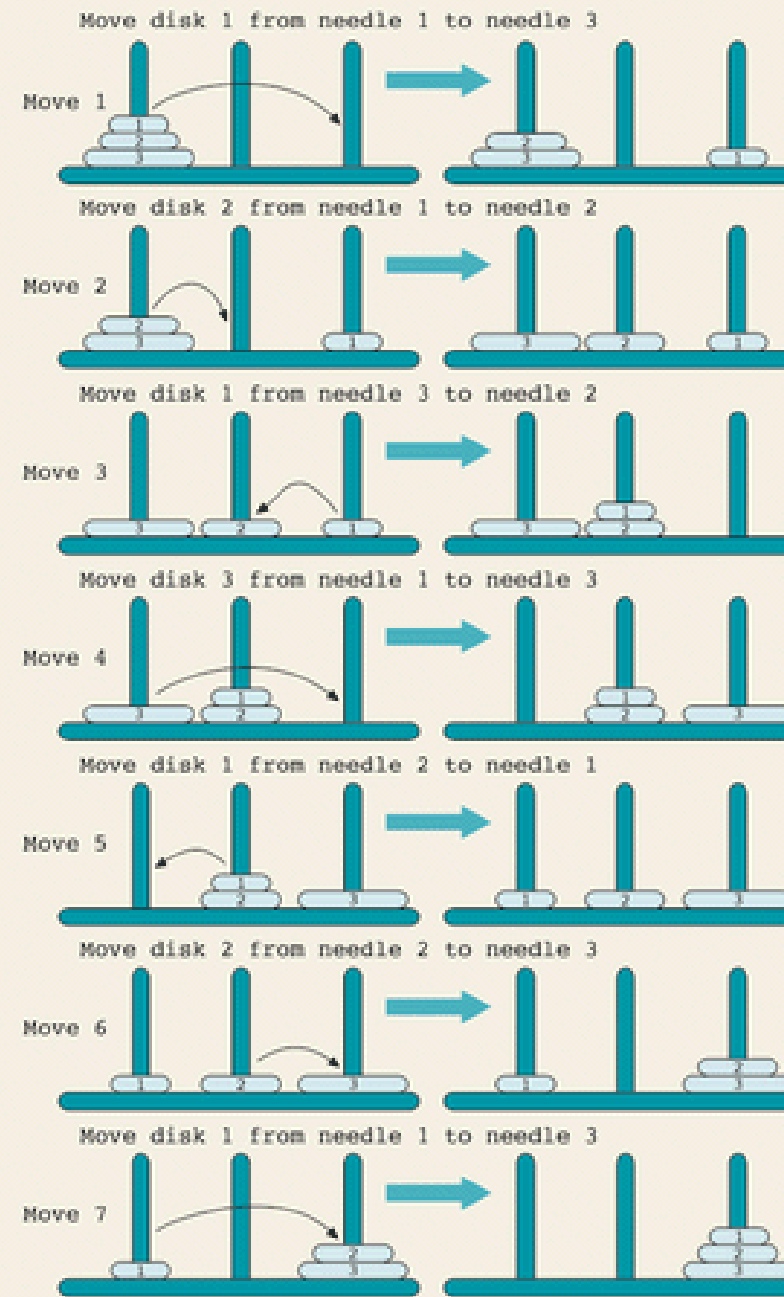


Figure 11-7 Solution of Tower of Hanoi problem with three disks

Complexity: Tower of Hanoi

- 3-disk problem: $2^3 - 1 = 7$ moves
- 64-disk problem: $2^{64} - 1$.
 - $2^{10} = 1024 \approx 1000 = 10^3$,
 - $2^{64} = 2^4 * 2^{60} \approx 2^4 * 10^{18} = 1.6 * 10^{19}$
- One year $\approx 3.2 * 10^7$ seconds

Complexity: Tower of Hanoi

- The wizard's speed = one disk / second

$$1.6 * 10^{19} = 5 * 3.2 * 10^{18} =$$

$$5 * (3.2 * 10^7) * 10^{11} =$$

$$(3.2 * 10^7) * (5 * 10^{11})$$

Complexity: Tower of Hanoi

- The time required to move all 64 disks from needle 1 to needle 3 is roughly $5 * 10^{11}$ years.
- It is estimated that our universe is about 15 billion = $1.5 * 10^{10}$ years old.

$$5 * 10^{11} = 50 * 10^{10} \approx 33 * (1.5 * 10^{10}).$$

Complexity: Tower of Hanoi

- Assume: a computer with 1 billion = 10^9 moves/second.
 - $\text{Moves/year} = (3.2 * 10^7) * 10^9 = 3.2 * 10^{16}$
- To solve the problem for 64 disks:
 - $2^{64} \approx 1.6 * 10^{19} = 1.6 * 10^{16} * 10^3 =$
 $(3.2 * 10^{16}) * 500$
 - 500 years for the computer to generate 2^{64} moves at the rate of 1 billion moves per second.

Complexity



- Why worry about complexity of algorithms?
 - because a problem may be solvable in principle but may take too long to solve in practice
- How can we evaluate the complexity of algorithms?
 - through asymptotic (tiệm cận) analysis, i.e., estimate time (or number of operations) necessary to solve an instance of size n of a problem when n tends towards infinity
 - See AIMA, Appendix A.

Complexity of Algorithms

- $T(n)$ is $O(f(n))$ means there exists n_0, k such that for all $n > n_0$ $T(n) \leq kf(n)$:
 - N = input size
 - $T(n)$ = total number of step of the algorithm
- Independent of the implementation, compiler, ...
- Asymtotic analysis: For large n , an $O(n)$ algorithm is better than an $O(n^2)$ algorithm.
- $O()$ abstract over constant factors:
 - $T(100n + 1000)$ is better than $T(n^2 + 1)$ only for $n > 110$
- $O()$ notation is a good compromise between precision and easy of analysis

Remember: Implementation of search algorithms

```
Function General-Search(problem, Queuing-Fn) returns a solution, or failure
  nodes ← make-queue(make-node(initial-state[problem]))
  loop do
    if nodes is empty then return failure
    node ← Remove-Front(nodes)
    if Goal-Test[problem] applied to State(node) succeeds then return node
    nodes ← Queuing-Fn(nodes, Expand(node, Operators[problem]))
  end
```

Queuing-Fn(*queue, elements*) is a queuing function that inserts a set of elements into the queue and **determines the order of node expansion**. Varieties of the queuing function produce varieties of the search algorithm.

Types of search strategies



- Uninformed/blind search
 - Can only generate successors, sum up what has happened so far, and distinguish goals from non-goals
- Informed/heuristic search:
 - Strategies that know whether one non-goal state is more promising than another
- Adversarial Search



Uninformed/Blind search strategies

Uninformed/Blind search strategies



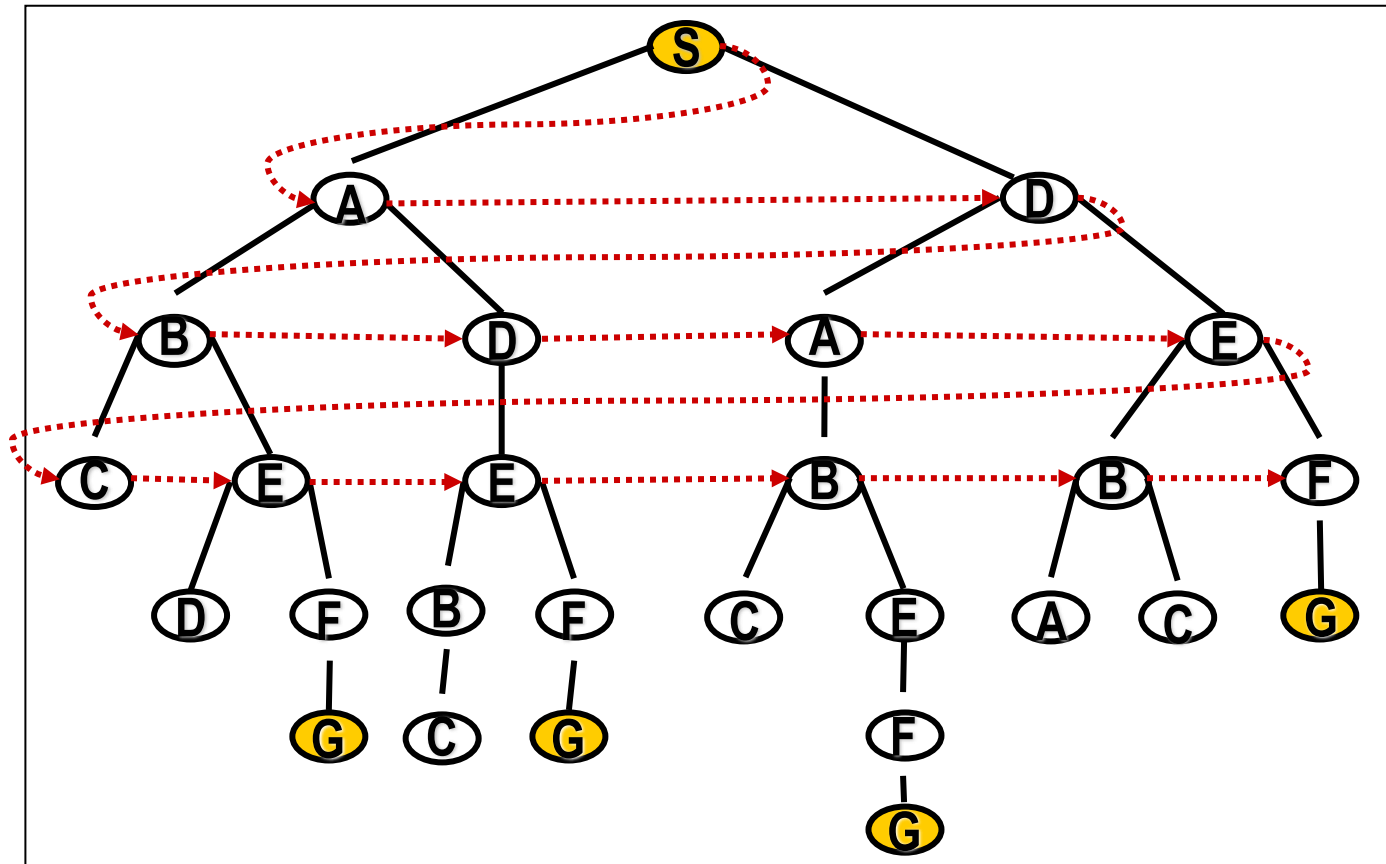
- Breadth-first search)
- Depth-first search)
- Depth-limited search
- Iterative deepening depth-first search
- Uniform-cost Search

Breadth-first search - Idea



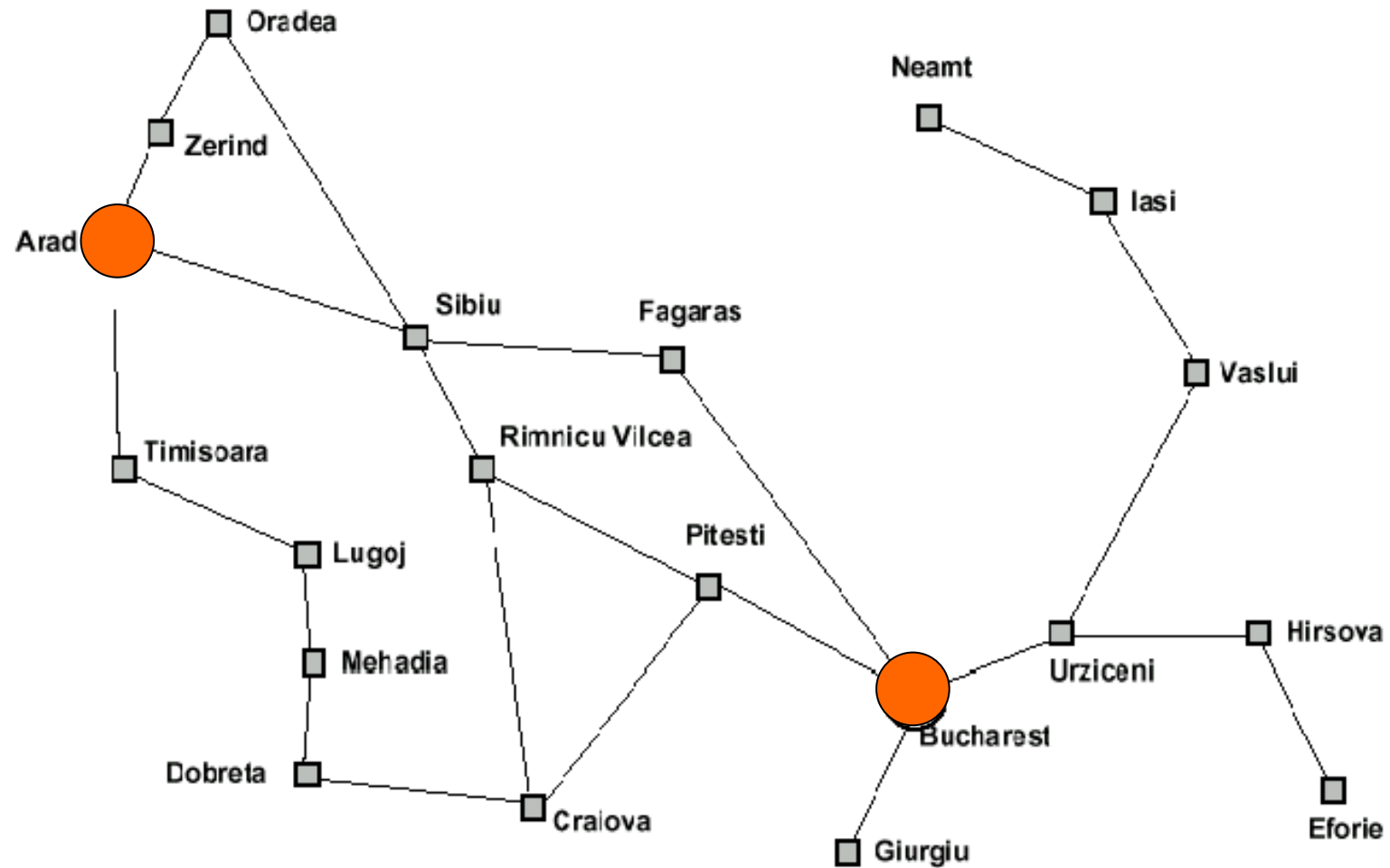
- **Strategy:** Expand shallowest unexpanded node in the tree
 - All nodes at a given level of the search tree are expanded before any other node is expanded. “Horizontal”, level-by-level search
- Can be implemented by using a **FIFO queue** for the frontier set

Breadth-first search

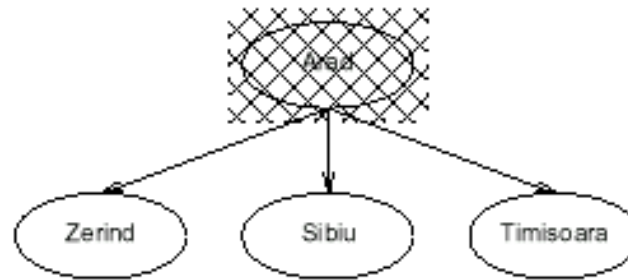


Move downwards,
level by level,
until goal is
reached.

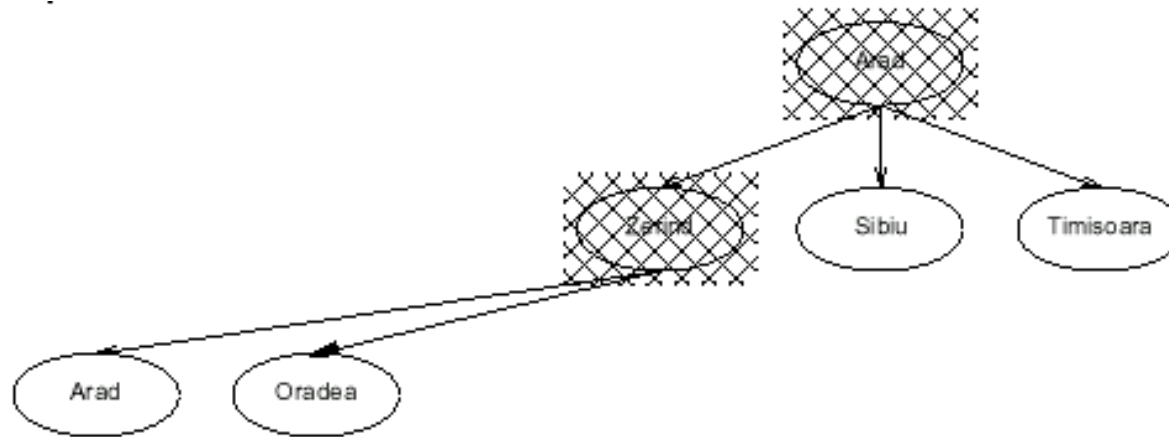
Example: Traveling from Arad To Bucharest



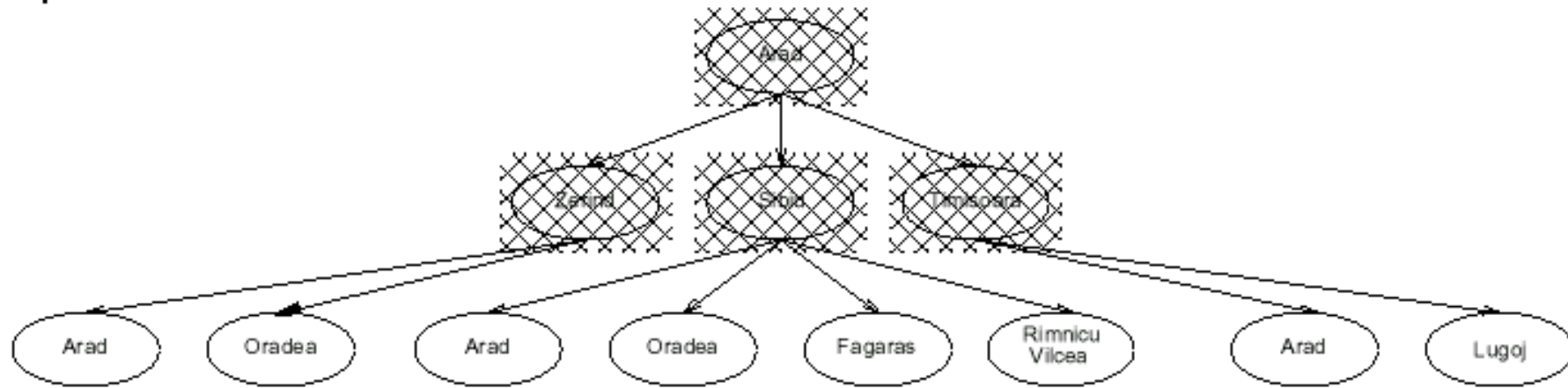
Breadth-first search



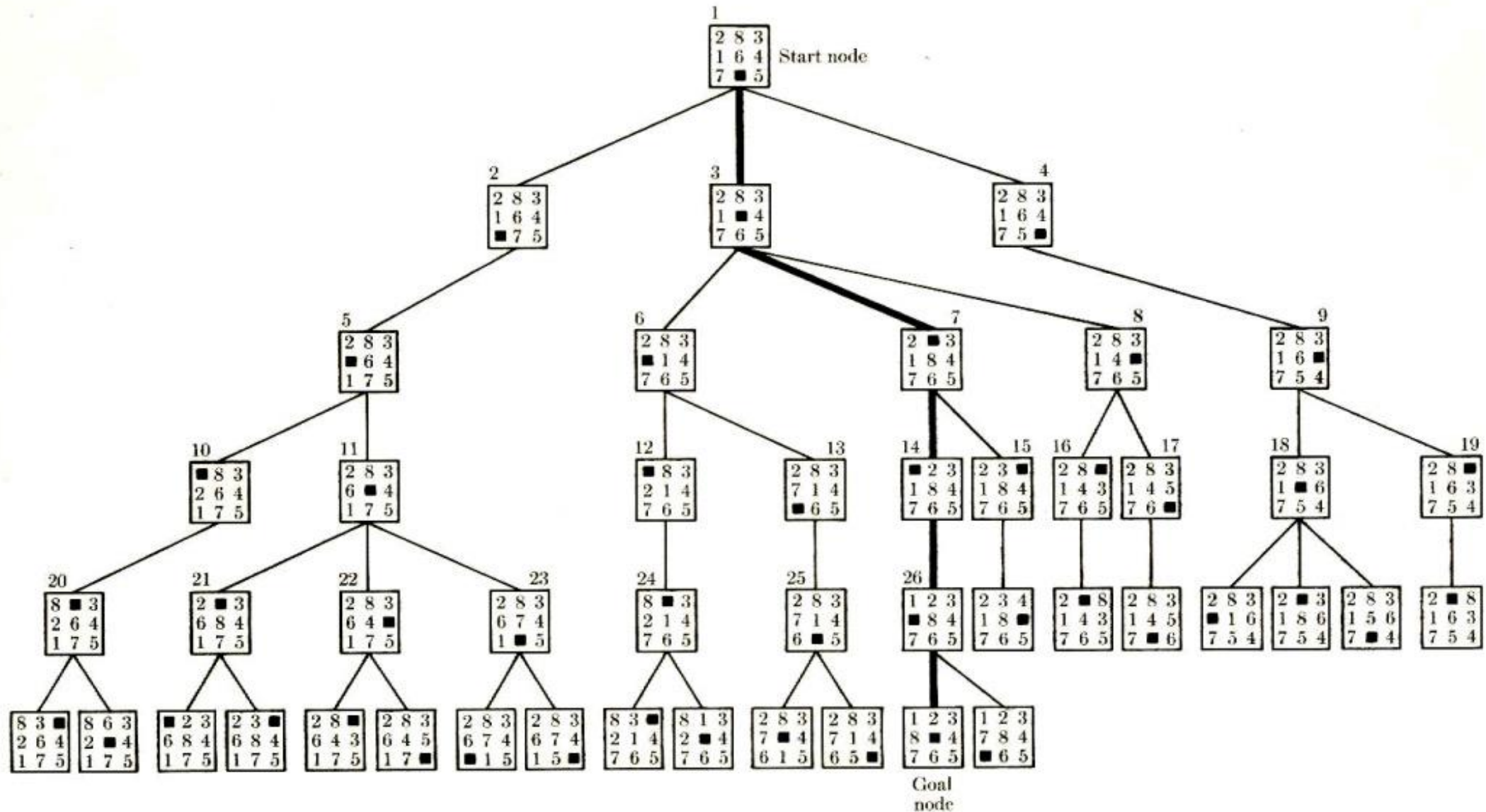
Breadth-first search



Breadth-first search



BFS Graph Search for 8-Puzzle



Breadth-first search (BFS)

From the general search algorithm we have:

1. Initialize queue L containing only start state
2. **Loop do**
 - 2.1 **If** (L == Empty) **then**
 {failed search message; end}
 - 2.2 Remove u state from beginning of the queue L;
 - 2.3 **If** (u == Goal state) **then**
 {successful search message; end}
 - 2.4 **For** (each state v expand from u) **do**
 {Put v at the end of queue L;}

Properties of breadth-first search

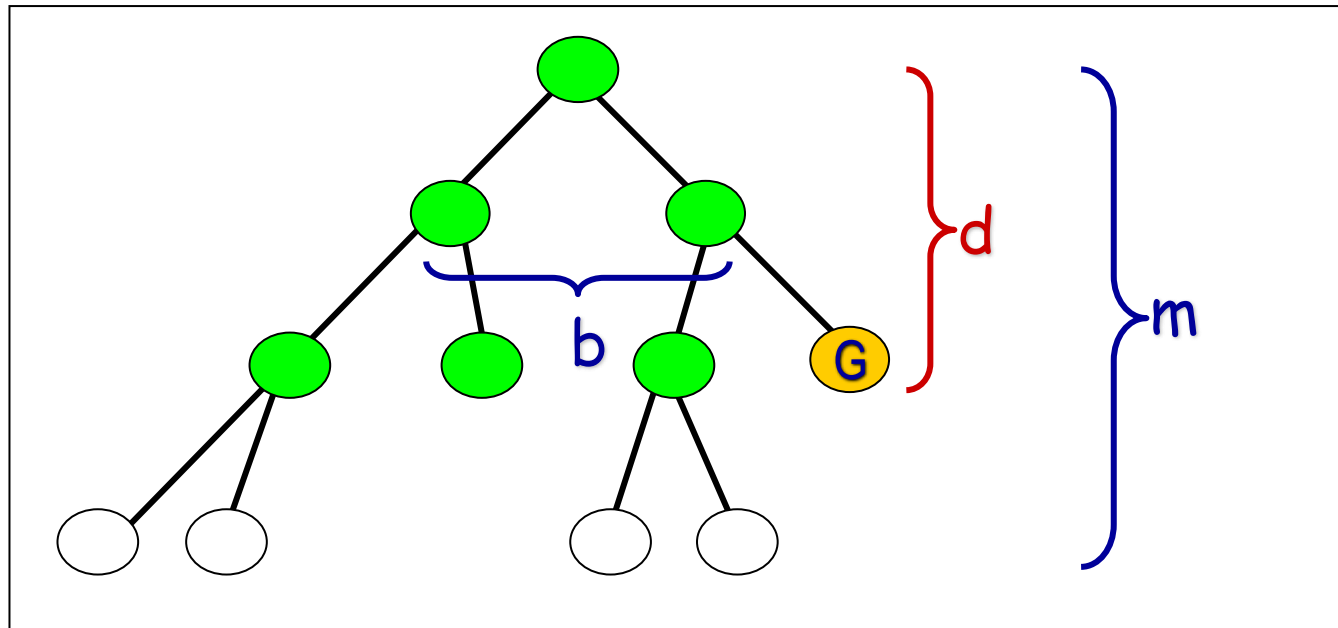
- Completeness:
 - Time complexity:
 - Space complexity:
 - Optimality:
-
- Search algorithms are commonly evaluated according to the following four criteria:
 - **Completeness:** does it always find a solution if one exists?
 - **Time complexity:** how long does it take as function of num. of nodes?
 - **Space complexity:** how much memory does it require?
 - **Optimality:** does it guarantee the least-cost solution?
 - Time and space complexity are measured in terms of:
 - b – max branching factor of the search tree
 - d – depth of the least-cost solution
 - m – max depth of the search tree (may be infinity)

Properties of breadth-first search

- Completeness: Yes, if b is finite
- Time complexity: $1+b+b^2+\dots+b^d = O(b^d)$, i.e., exponential in d
- Space complexity: $O(b^d)$ (see following slides)
- Optimality: Yes (assuming cost = 1 per step)

Time complexity of breadth-first search

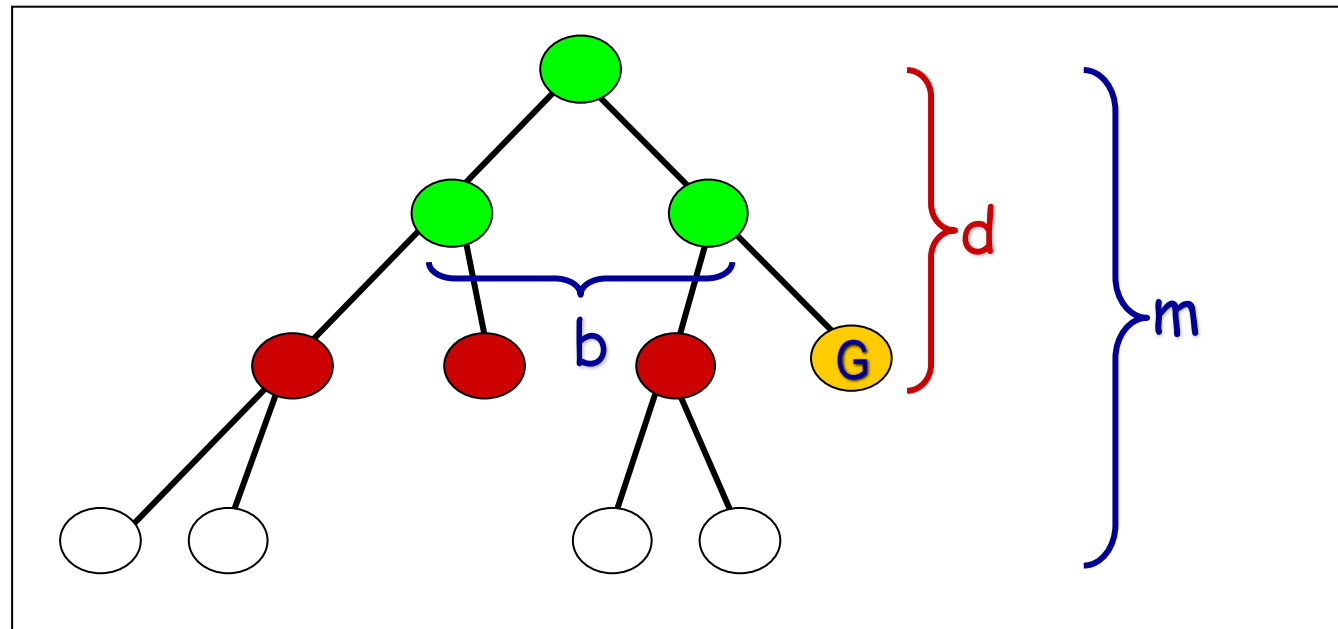
- If a goal node is found on depth d of the tree, all nodes up till that depth are created.



- Thus: $O(b^d)$

Space complexity of breadth-first

- Largest number of nodes in QUEUE is reached on the level d of the goal node.



- QUEUE contains all ● and G nodes. (Thus: 4) .
- In General: b^d

Examples

- Assuming $b=10$, checking 1000 states takes 1 second, storing one state takes 100 bytes

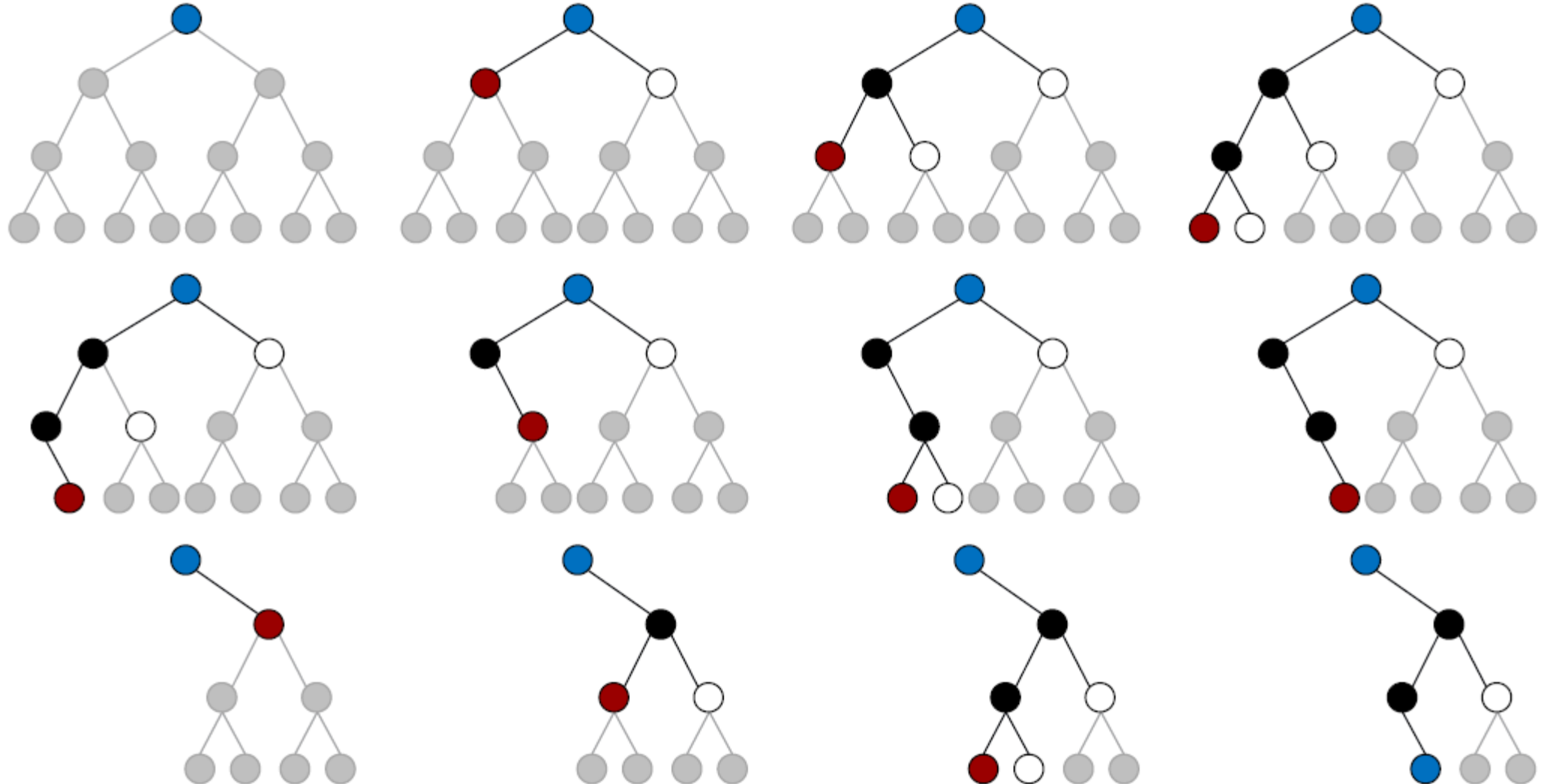
depth d	Time	Space
4	11 s	1 megabyte
6	18 s	111 megabyte
8	31 h	11 gigabyte
10	128 d	1terabyte
12	35 y	111 terabyte
14	3500 y	11 111 terabyte

Depth First Search - Idea

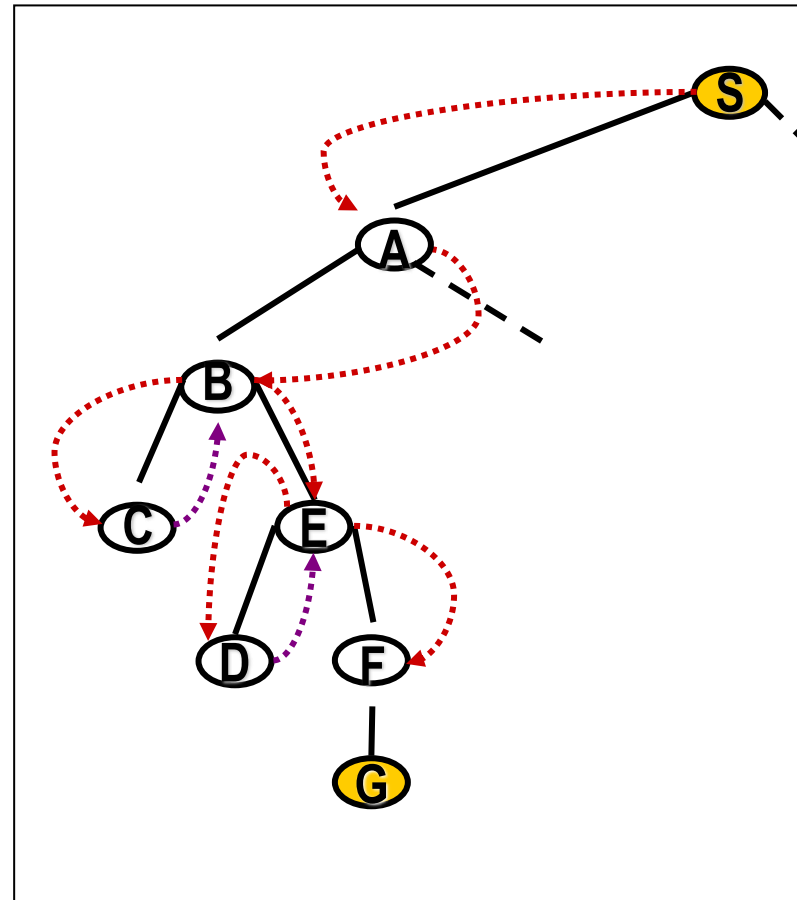


- **Strategy:** Expand deepest unexpanded node
- In other words, we will scan the search tree by branch from the root
- Can be implemented by using a **stack** for the frontier (**LIFO**).

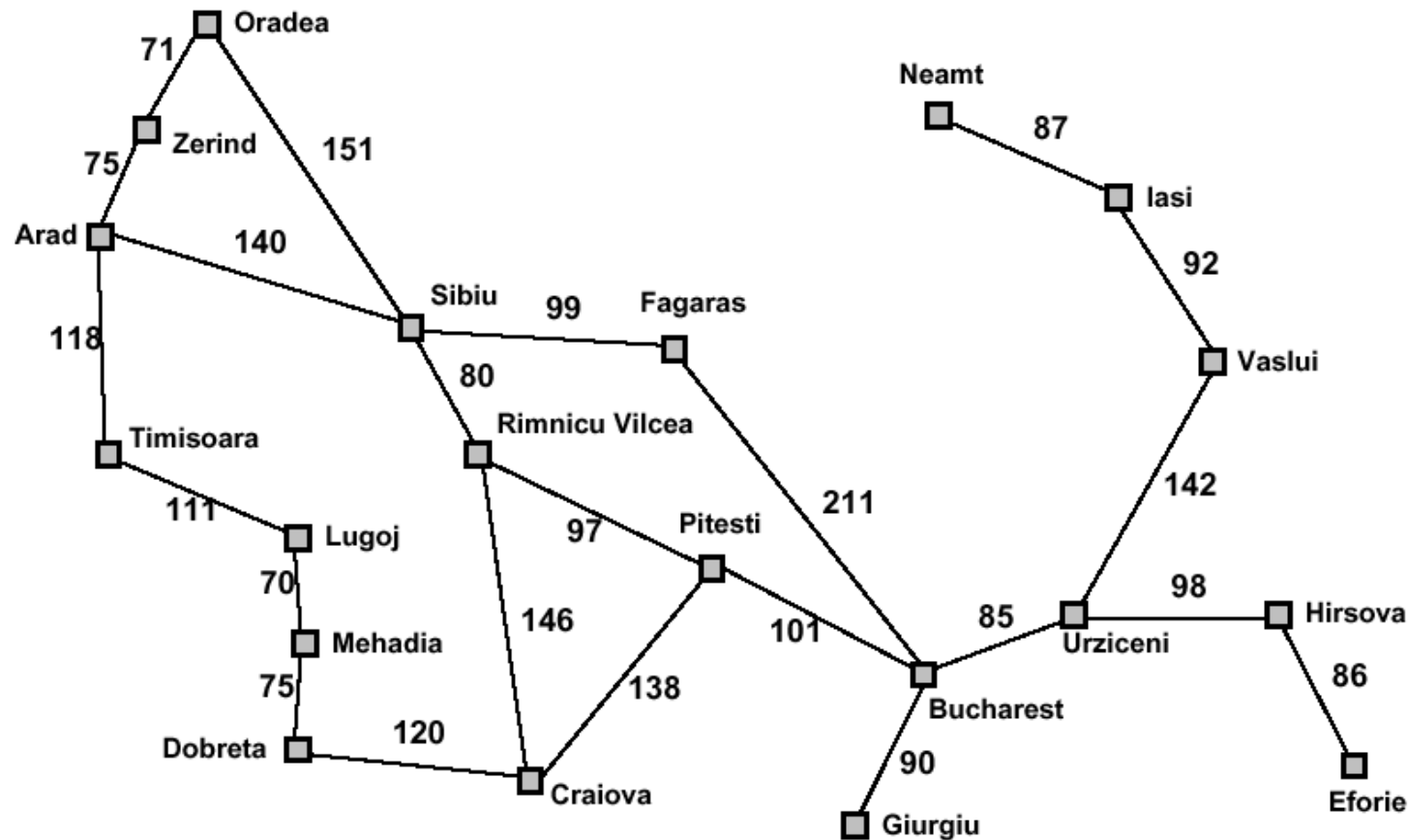
Depth First Search - Idea



Depth First Search



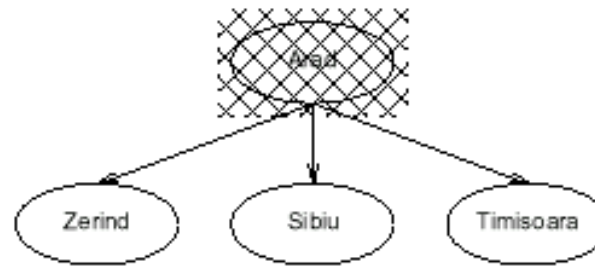
Romania with step costs in km



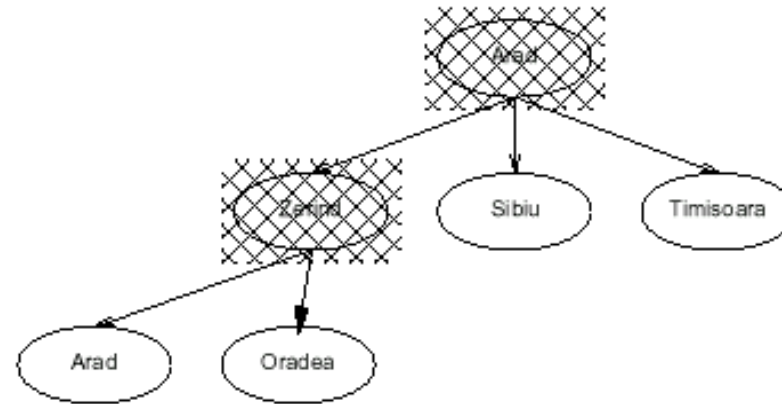
Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

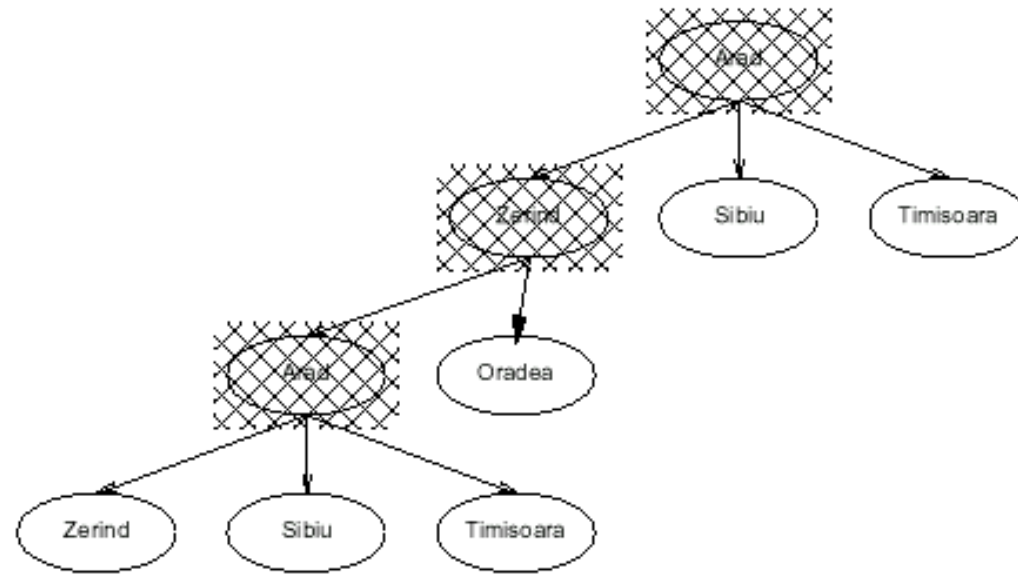
Depth-first search



Depth-first search

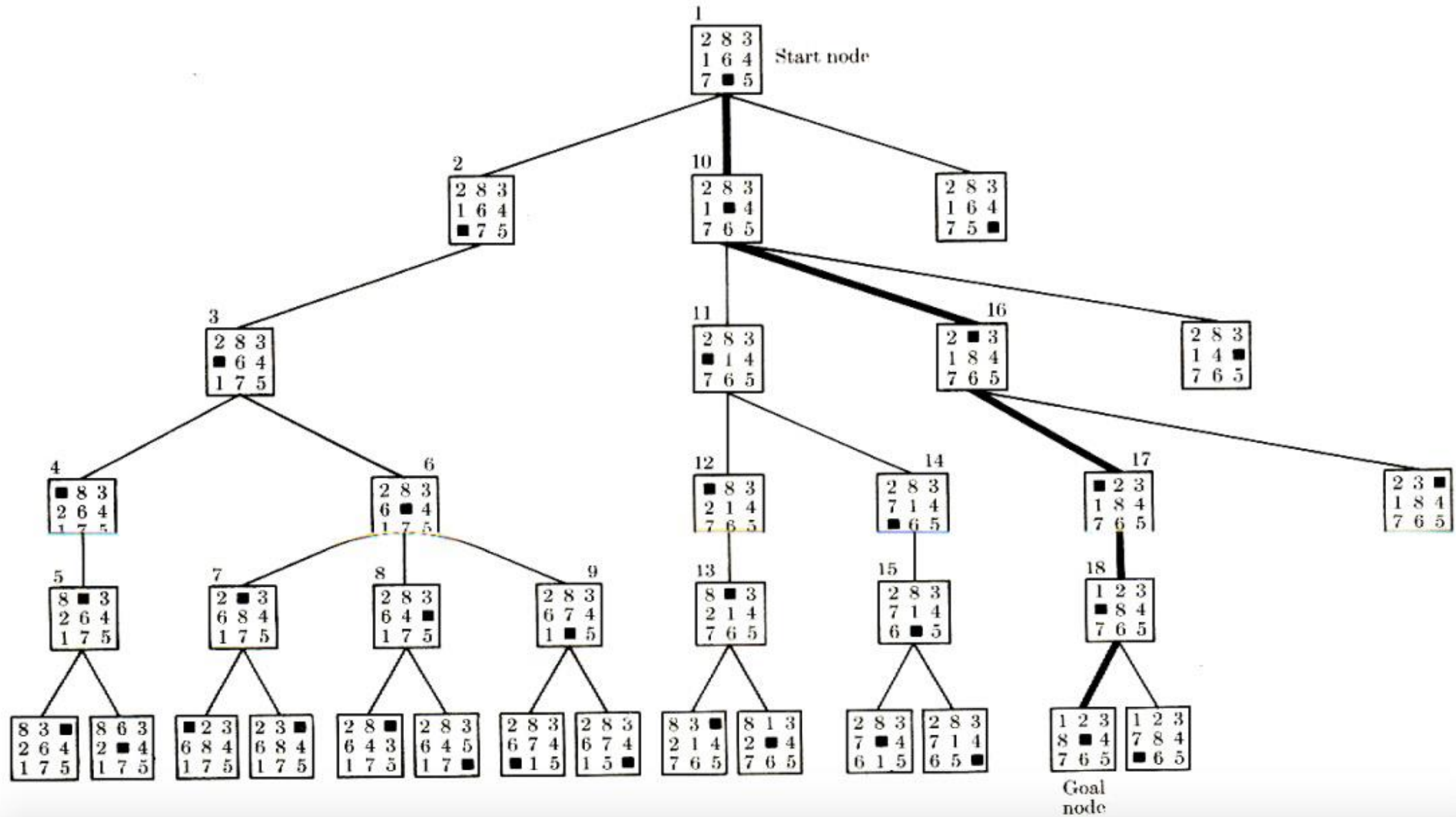


Depth-first search



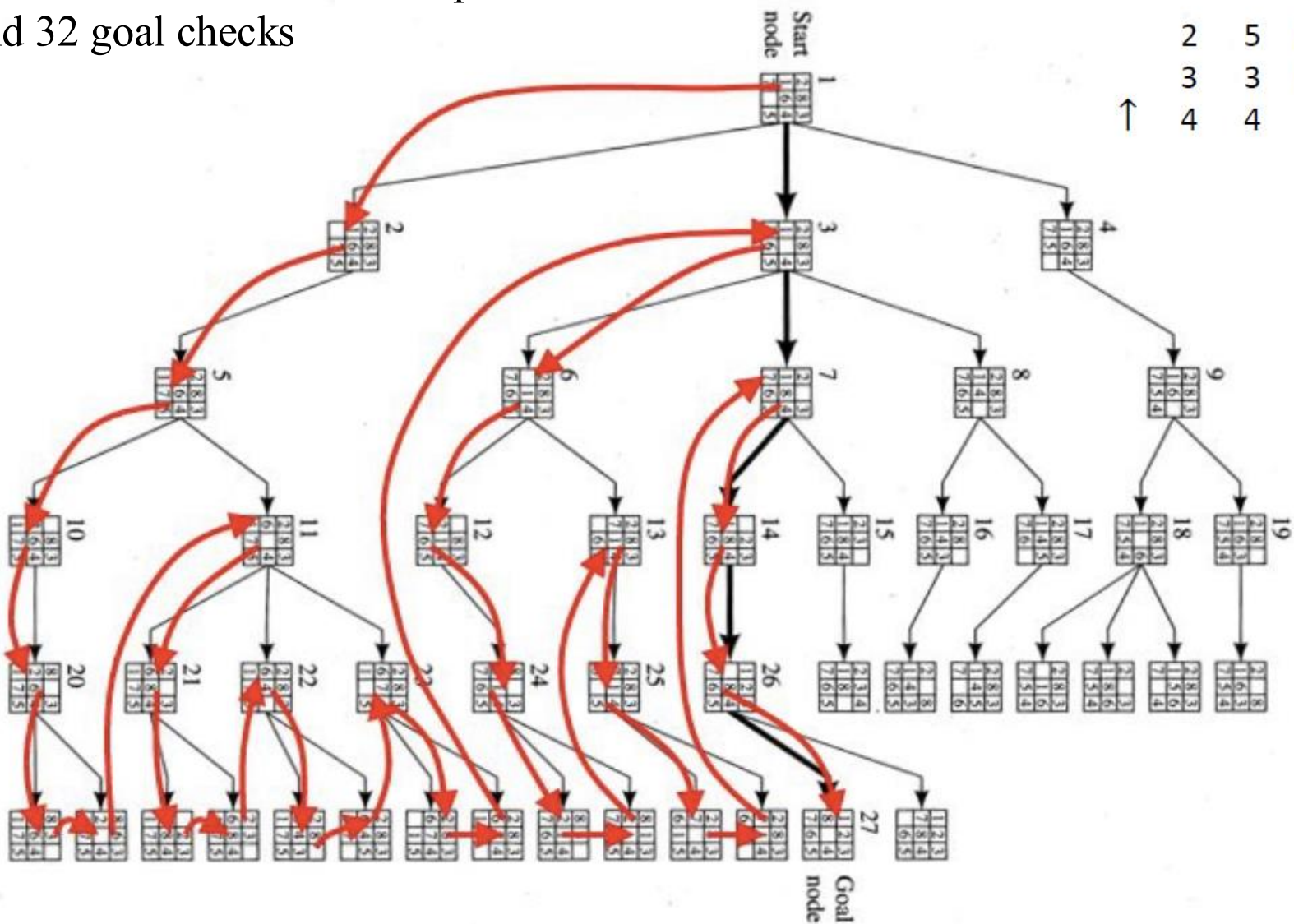
I.e., depth-first search can perform infinite cyclic excursions
Need a finite, non-cyclic search space (or repeated-state checking)

DFS for 8-Puzzle



DFS for 8-Puzzle

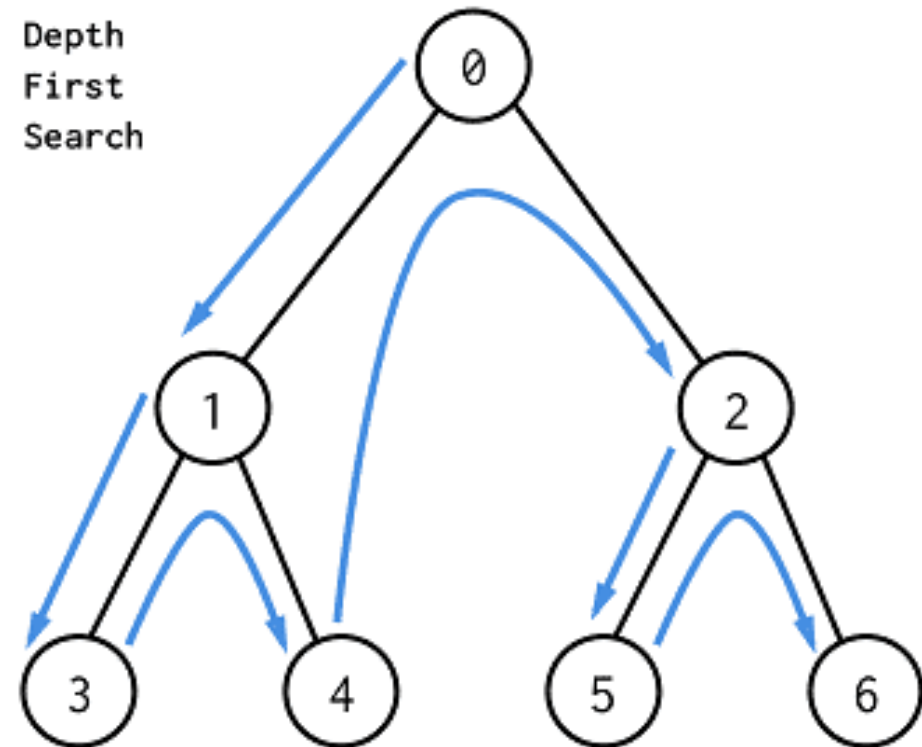
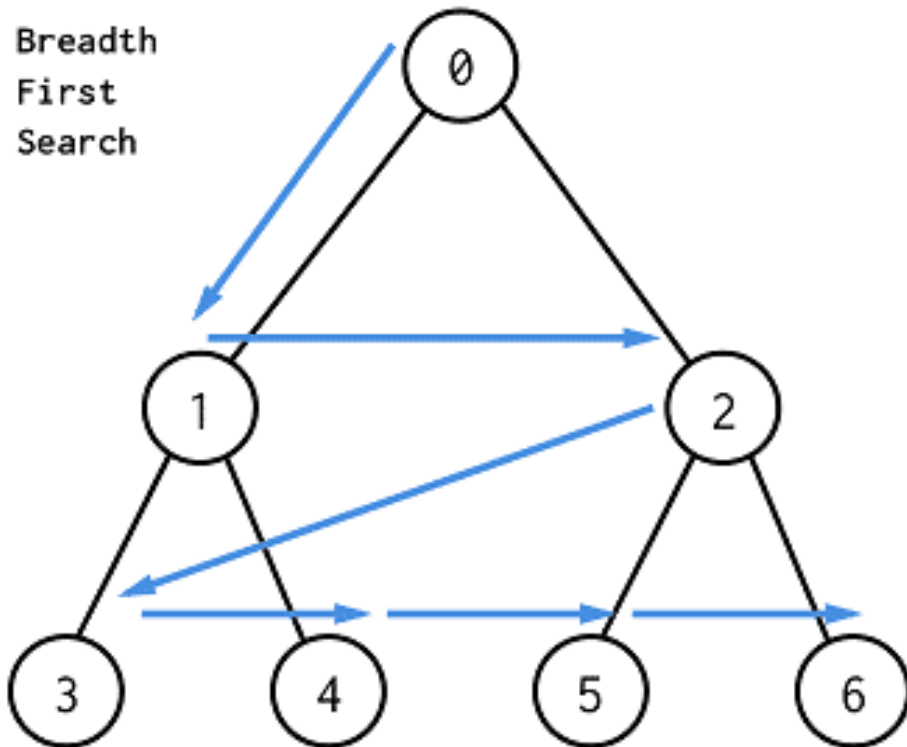
Goal found after 19 node expansions
and 32 goal checks



Stack:

2	5	10	20	20R	20L	11	21
3	3	11	11	20L	11	3	22
4	4	3	3	11	3	4	23
		4	4	3	4		3
				4			4

DFS vs. BFS



Depth-first search algorithm

From general search, deep-first search algorithm:

1. Initialize queue L containing only start state
2. **Loop do**
 - 2.1 **If** (L == Empty) **then**
 {failed search message; end}
 - 2.2 Remove u state from beginning of the stack L;
 - 2.3 **If** (u == Goal state) **then**
 {successful search message; end}
 - 2.4 **For** (each state v expanding from u) **do**
 {Put v at the beginning of stack L;}

Properties of depth-first search

- Completeness: No, fails in infinite state-space (yes if finite state space)
- Time complexity: $O(b^m)$
- Space complexity: $O(bm)$
- Optimality: No

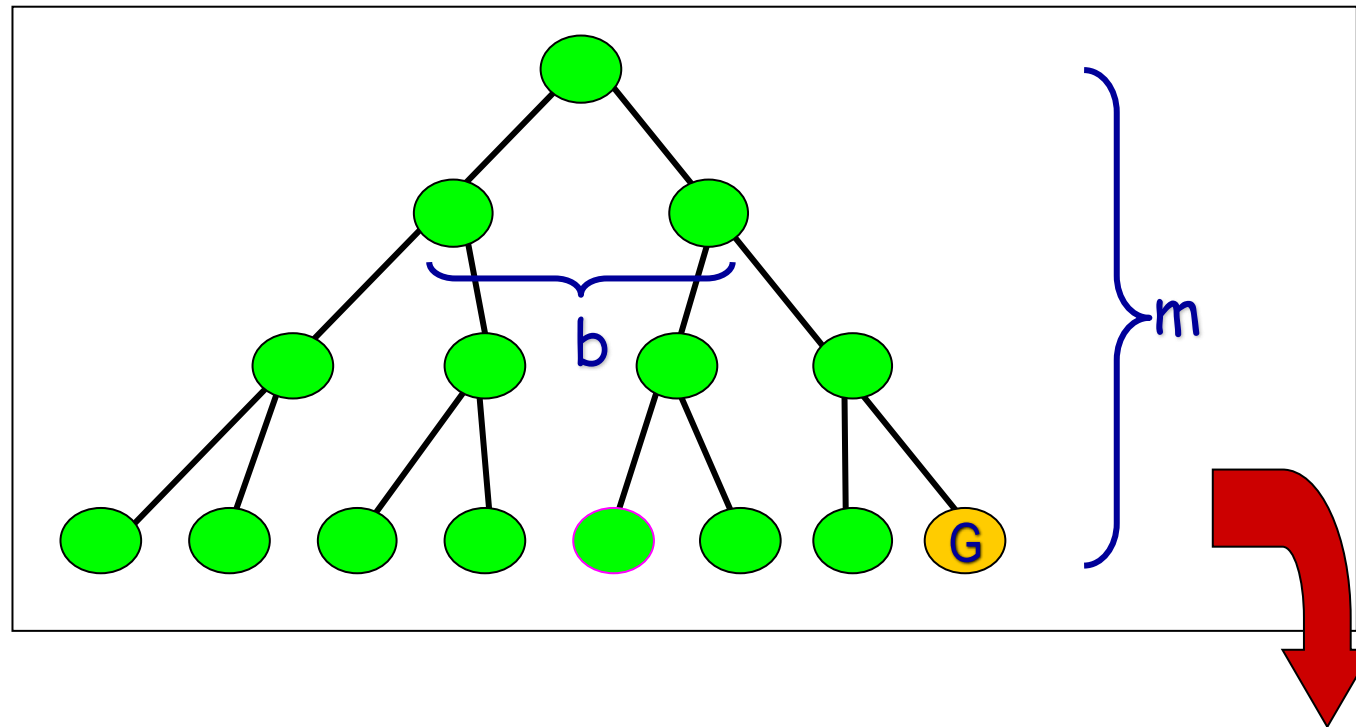
Remember:

b = branching factor

m = max depth of search tree

Time complexity of depth-first: details

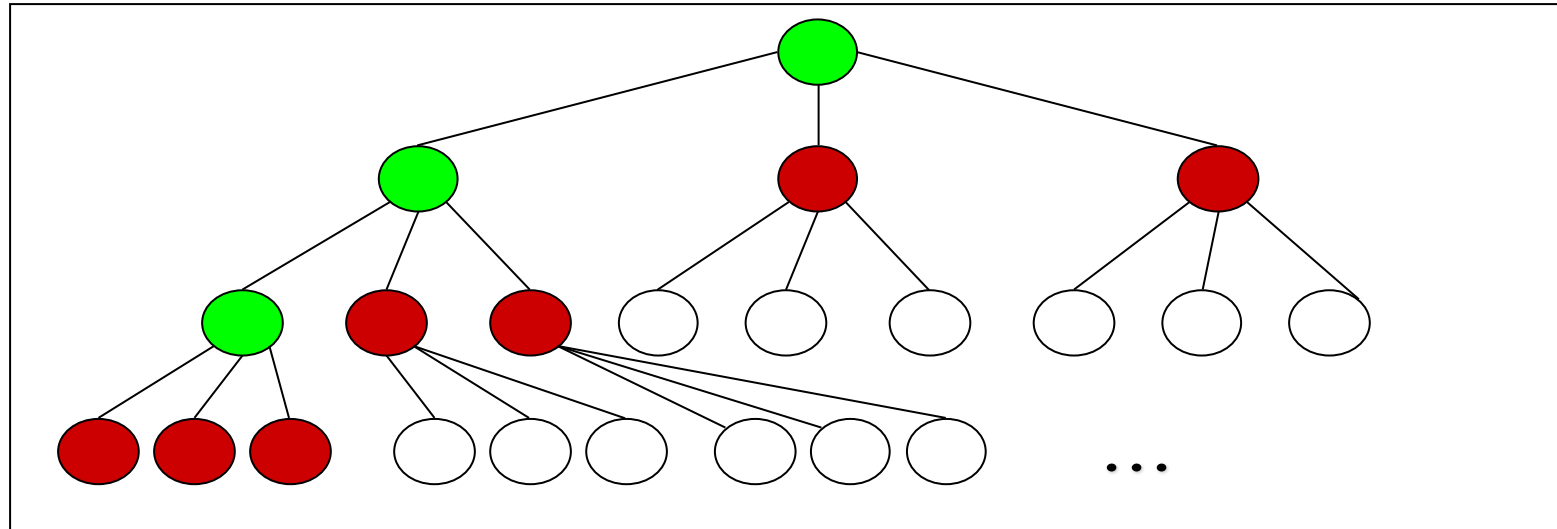
- In the worst case:
 - the (only) goal node may be on the right-most branch,



- Time complexity $= b^m + b^{m-1} + \dots + 1 = \frac{b^{m+1} - 1}{b - 1}$
- Thus: $O(b^m)$

Space complexity of depth-first

- Largest number of nodes in QUEUE is reached in bottom left-most node.
- Example: $m = 3, b = 3$:



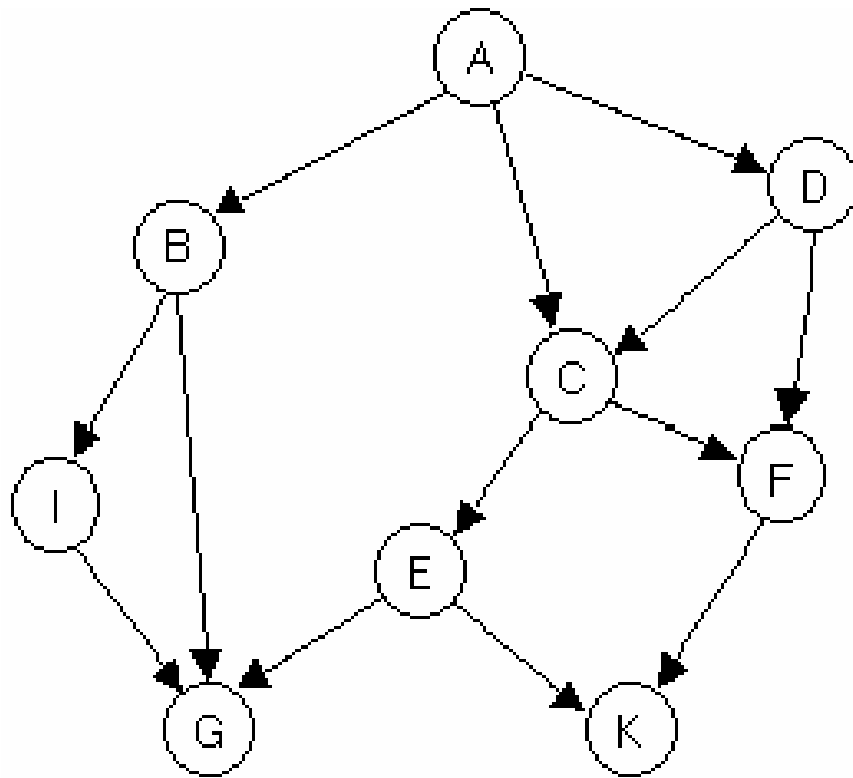
- QUEUE contains all  nodes. Thus: 7.
- In General: $((b-1) * m) + 1$
- Order: $O(m*b)$

Handling repeated states



- A search tree can contain many nodes corresponding to the same state – these states are called repeated states
- Search algorithms will waste a lot of time re-expanding the states we have already explored
- **To handle repeated states:**
 - Every time you expand a node, add that state to the **explored set**; do not put explored states on the frontier again
 - Every time you add a node to the frontier, check whether it already exists in the frontier with a higher path cost, and if yes, replace that node with the new one

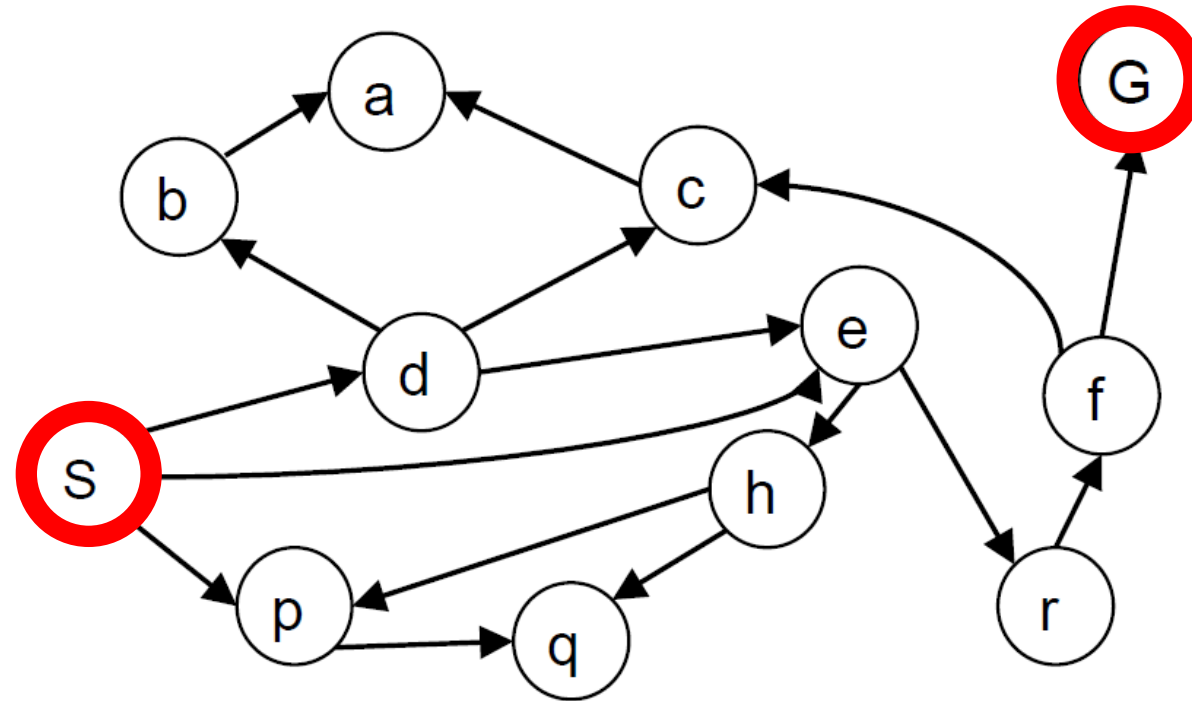
Exercise



Find path from A to K
by:

- Breadth-first
- Depth-first

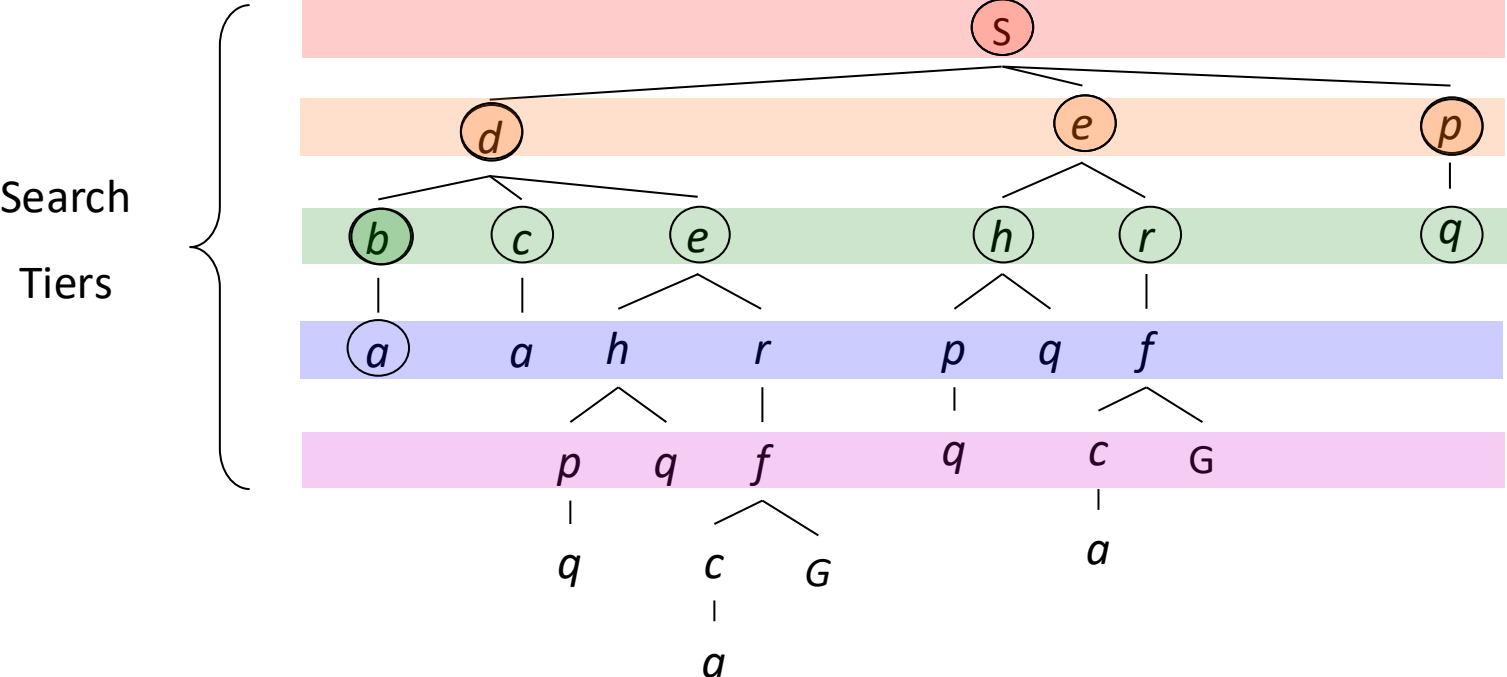
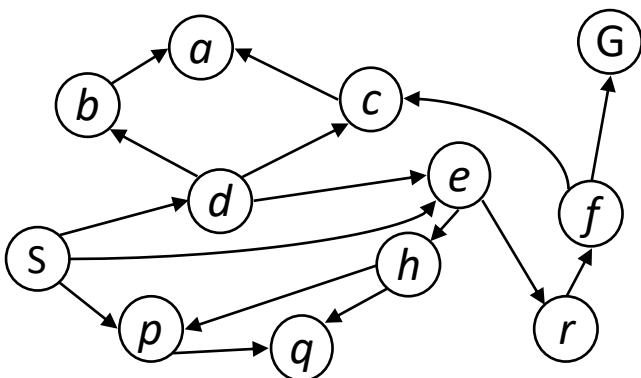
Exercise (2)



Breadth-First Search

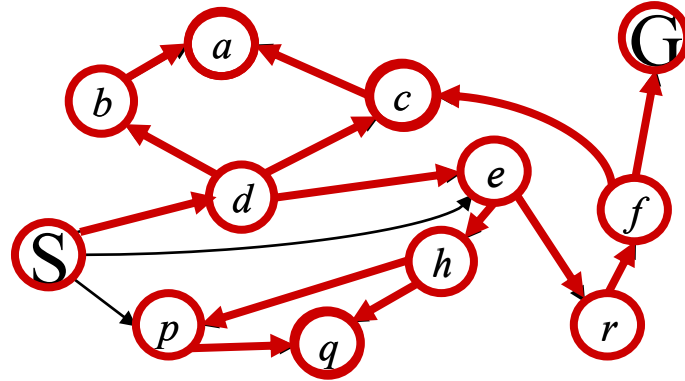
Strategy: expand a shallowest node first

Implementation:
Frontier is a FIFO queue



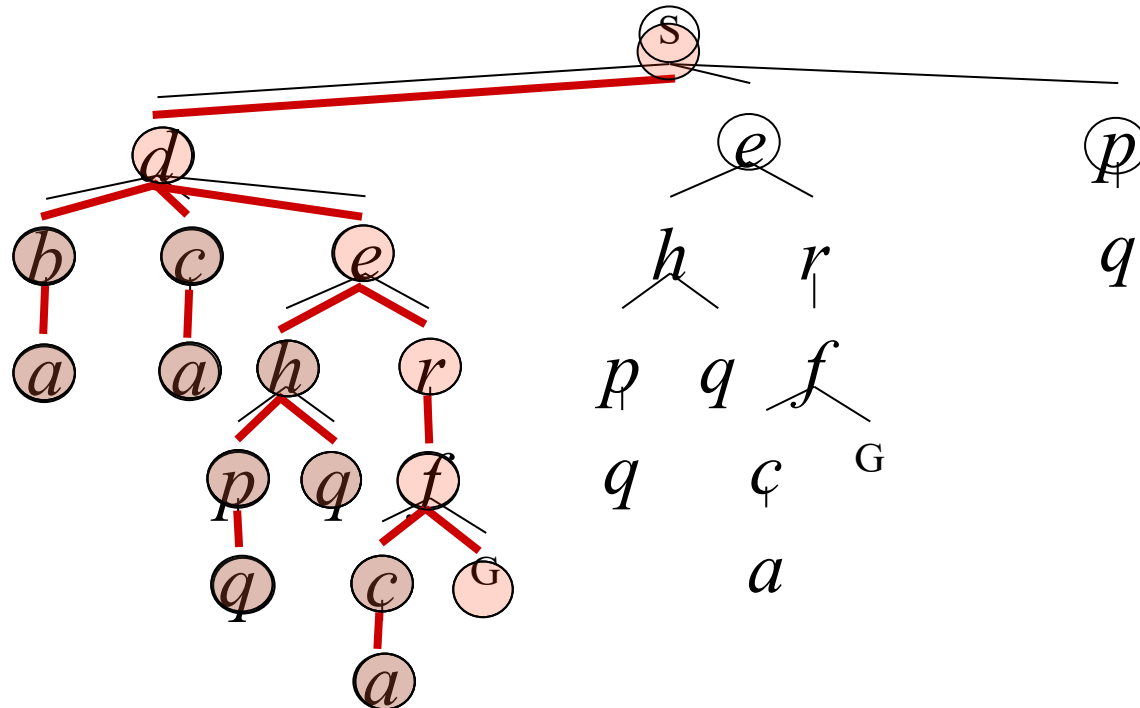
Depth-First Search

*Strategy:
expand a
deepest node
first*



Implementation

: Frontier is a LIFO stack



Quiz: DFS vs BFS

- When will BFS outperform DFS?
- When will DFS outperform BFS?

Depth-limited search

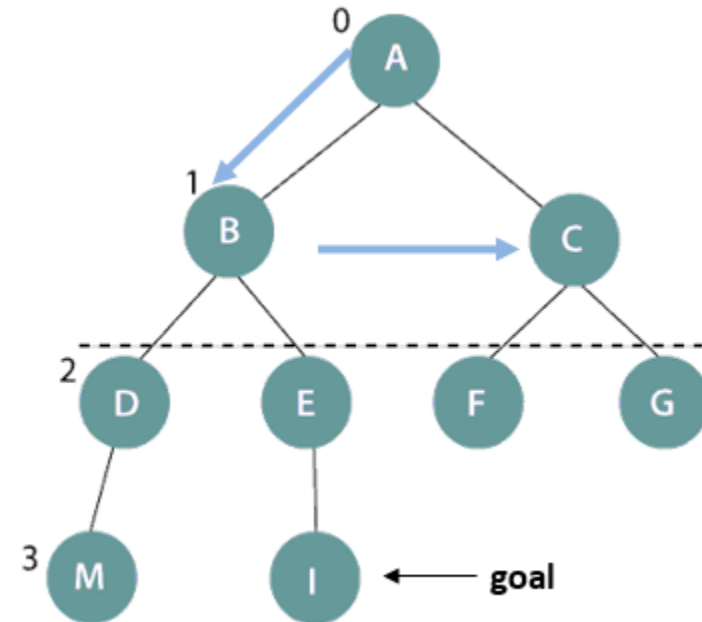
Is a depth-first search with depth limit l

Implementation:

- In the above figure, the **depth-limit is 1**.
- Depth-first search is a special case of depth-limited search.

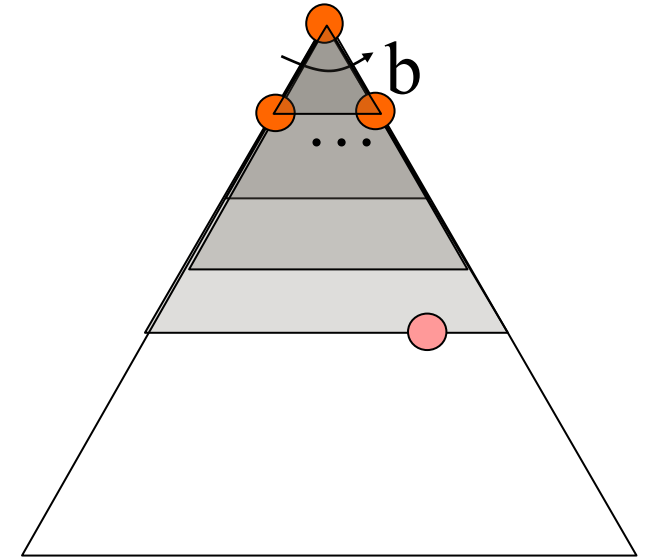
Complete: if cutoff chosen appropriately then it is guaranteed to find a solution.

Optimal: it does not guarantee to find the least-cost solution



Iterative Deepening

- Idea: get DFS's space advantage with BFS's time / shallow-solution advantages
 - Run a DFS with depth limit 1. If no solution...
 - Run a DFS with depth limit 2. If no solution...
 - Run a DFS with depth limit 3.
- Isn't that wastefully redundant?
 - Generally most work happens in the lowest level searched, so not so bad!



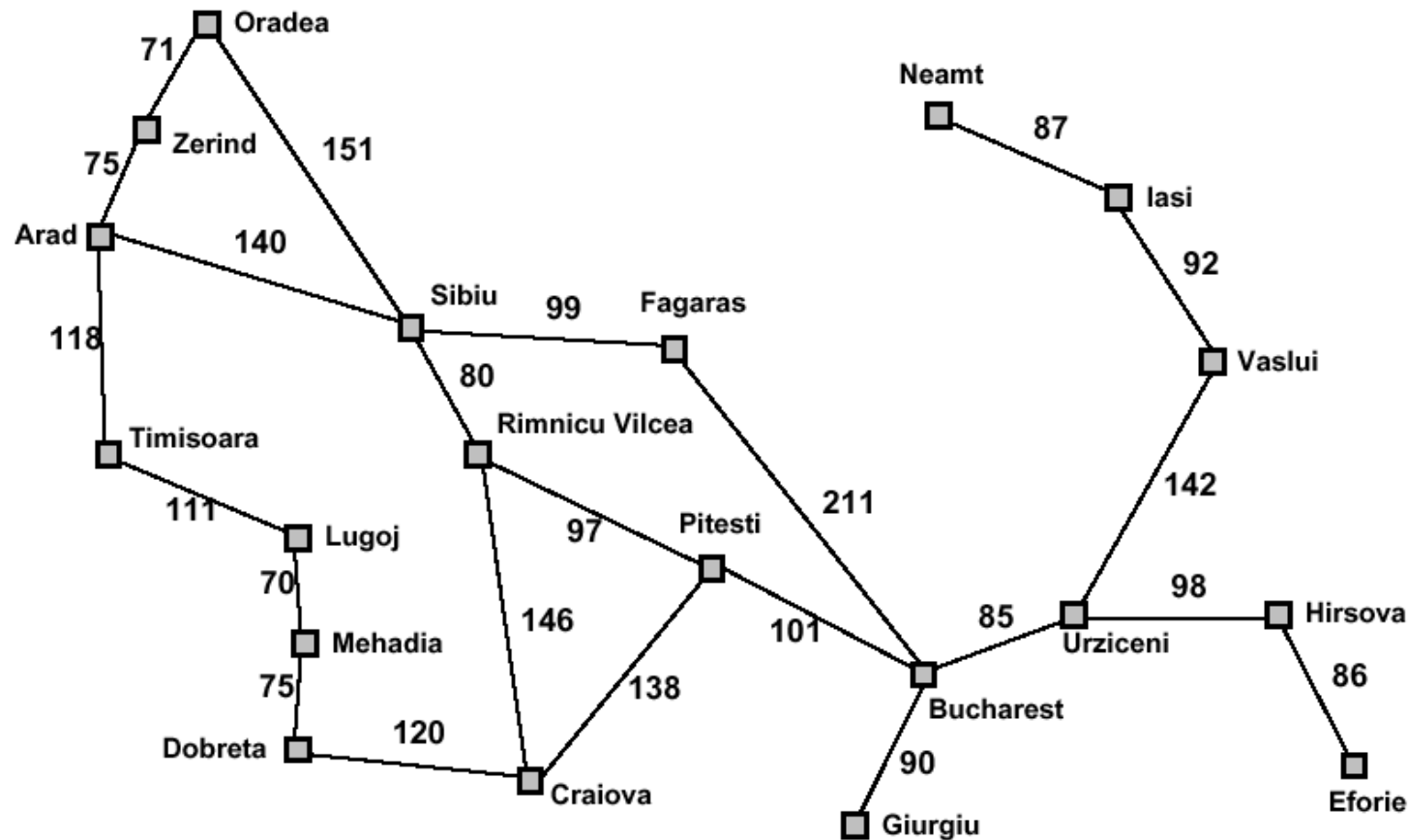
Iterative deepening search

```
Function Iterative-deepening-Search(problem) returns a solution,  
or failure  
  
  for depth = 0 to  $\infty$  do  
    result  $\leftarrow$  Depth-Limited-Search(problem, depth)  
    if result succeeds then return result  
end  
return failure
```

Combines the best of breadth-first and depth-first search strategies.

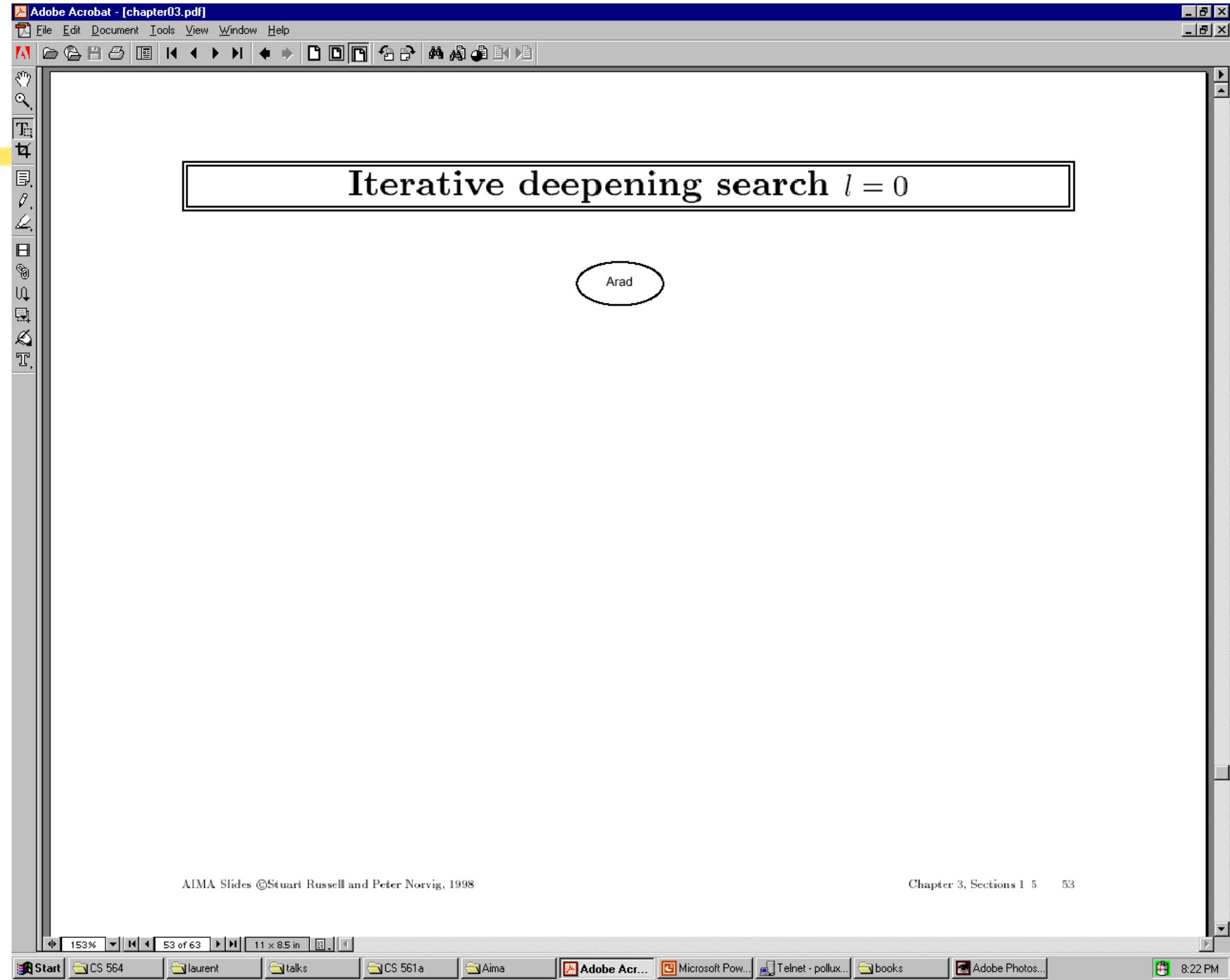
- Completeness: Yes,
- Time complexity: $O(b^d)$
- Space complexity: $O(bd)$
- Optimality: Yes, if step cost = 1

Romania with step costs in km

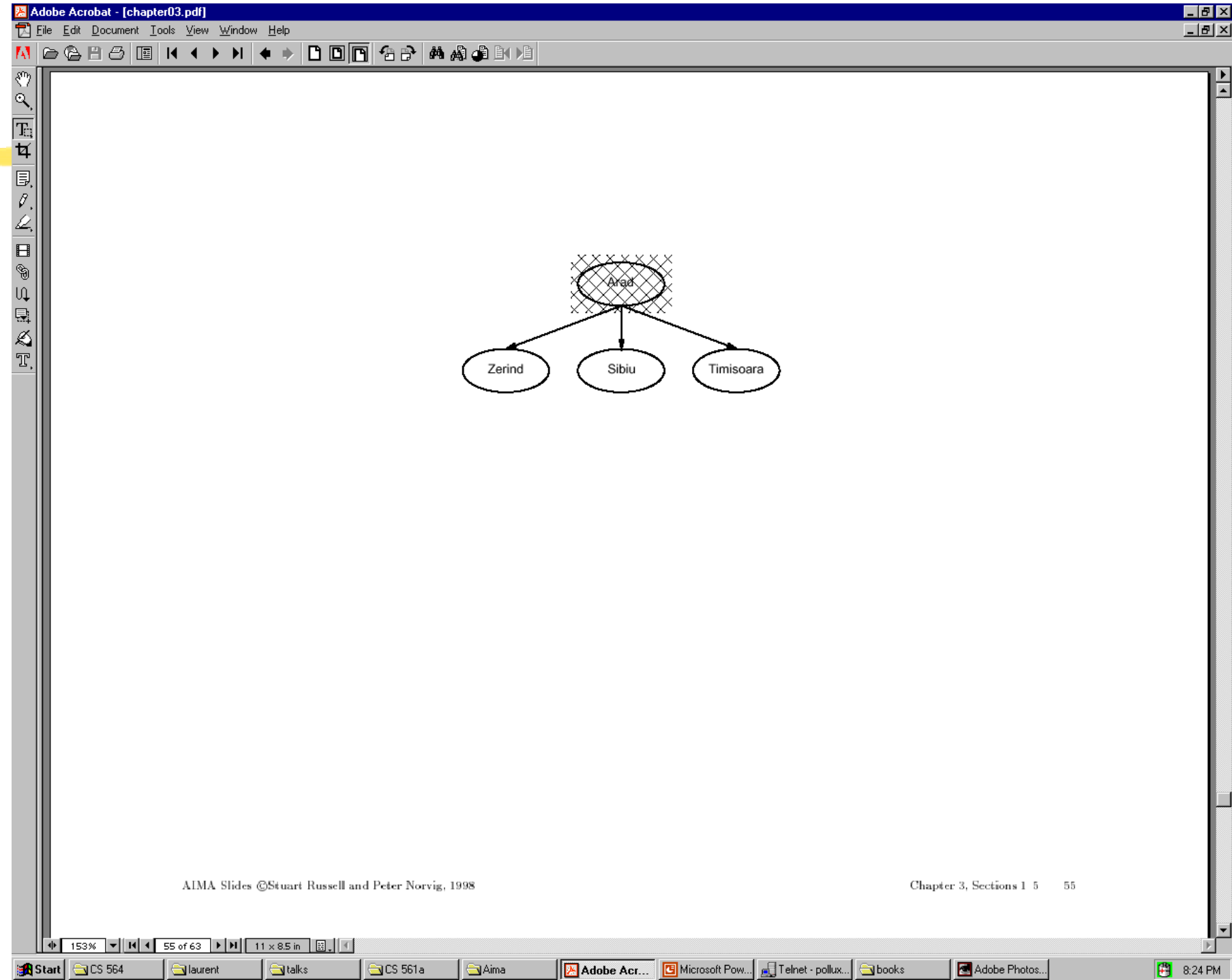


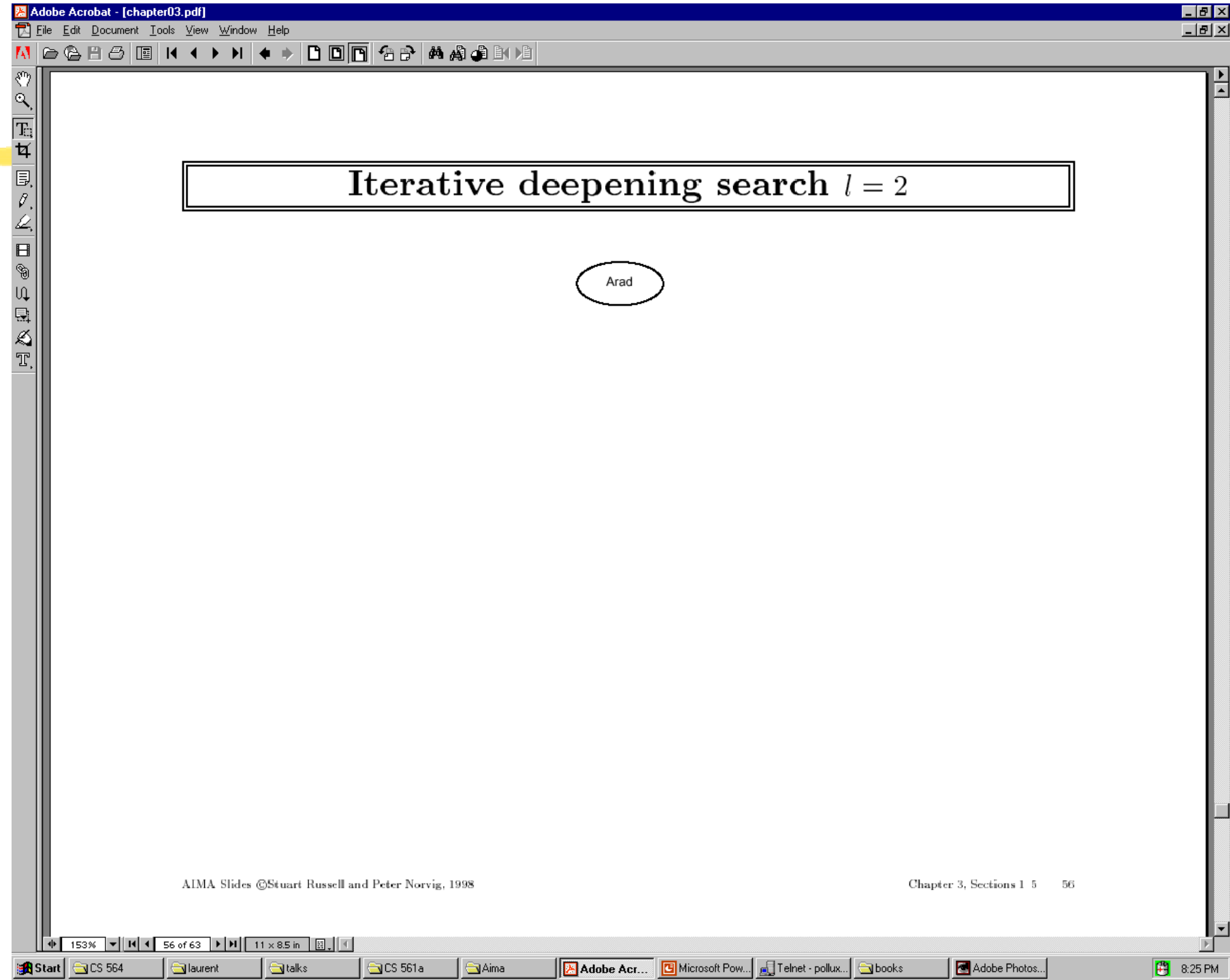
Straight-line distance
to Bucharest

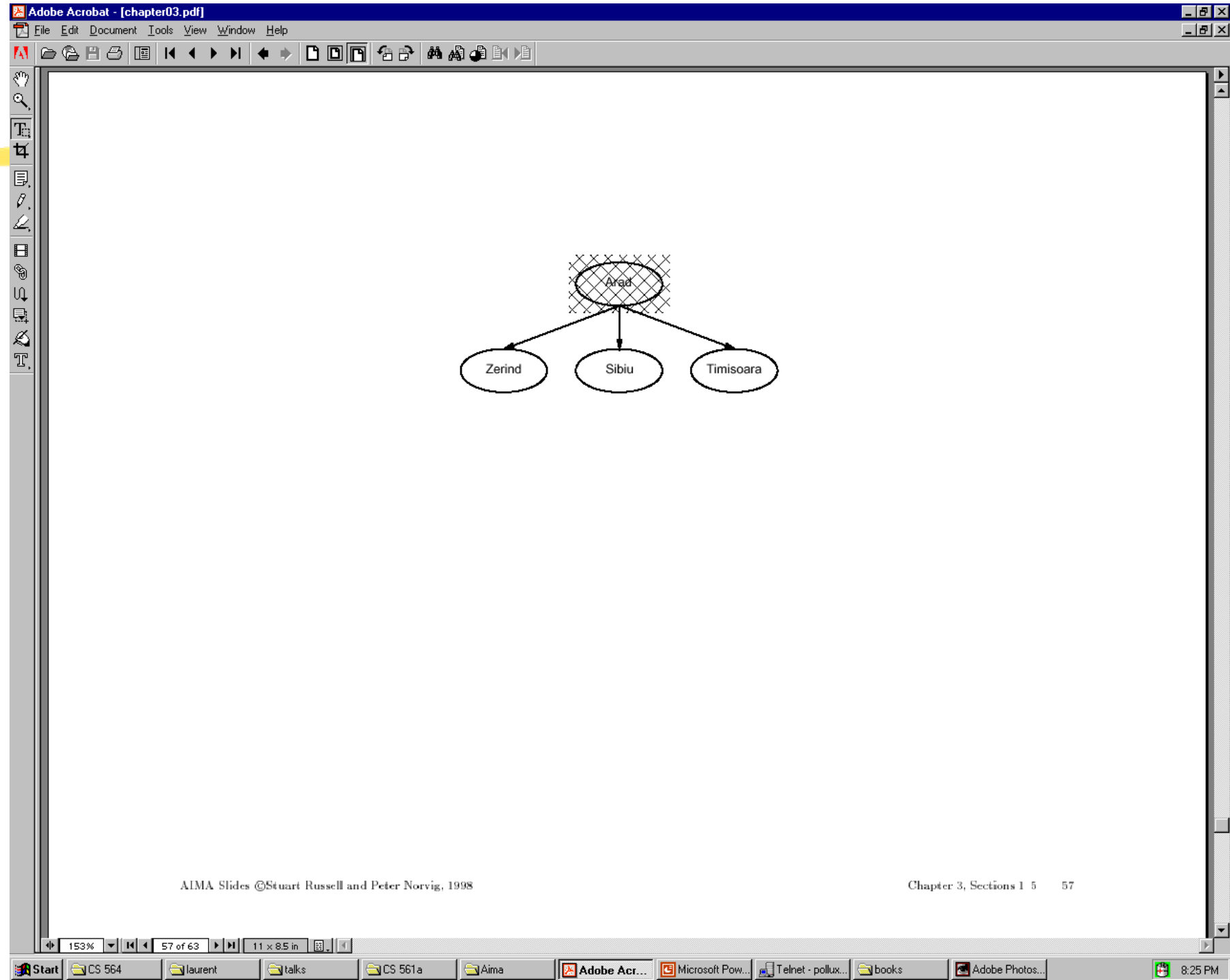
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

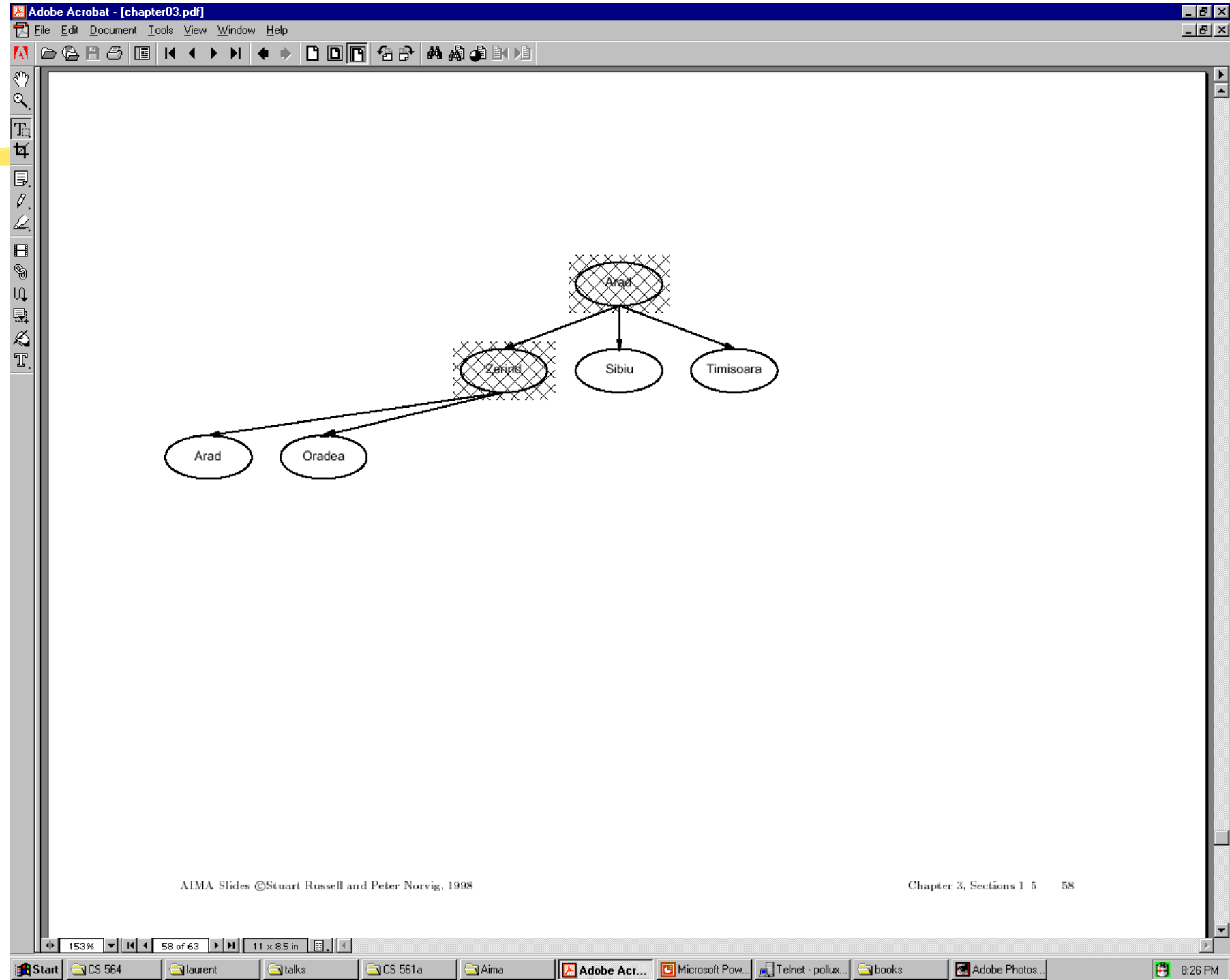


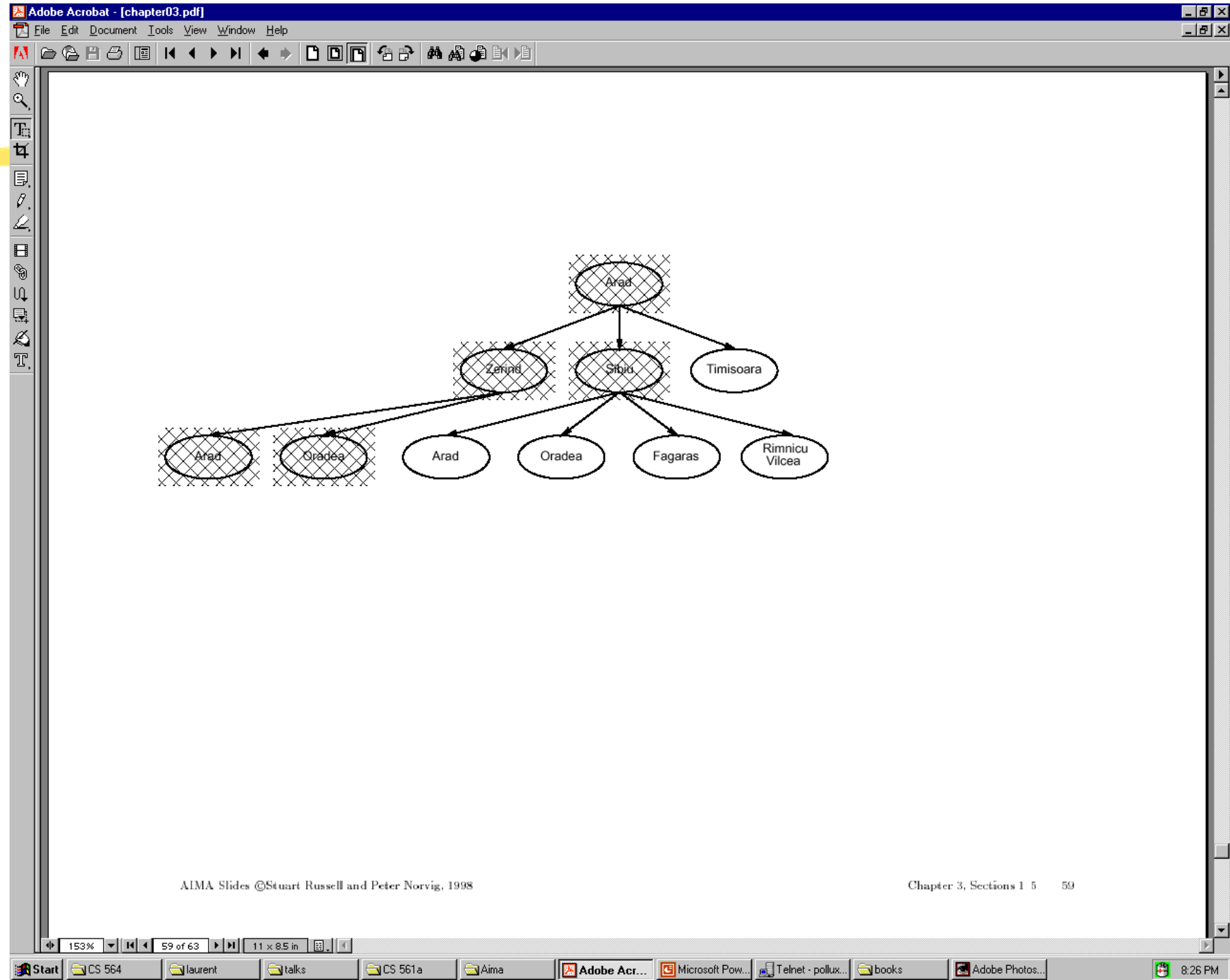












Adobe Acrobat - [chapter03.pdf]

File Edit Document Tools View Window Help

```
graph TD; Arad1((Arad)) --> Zerind((Zerind)); Arad1 --> Sibiu((Sibiu)); Arad1 --> Timisoara((Timisoara)); Zerind --> Arad2((Arad)); Zerind --> Oradea1((Oradea)); Sibiu --> Arad3((Arad)); Sibiu --> Oradea2((Oradea)); Sibiu --> Fagaras((Fagaras)); Sibiu --> BihorNicaa((Bihor/Nicea)); Timisoara --> Arad4((Arad)); Timisoara --> Lugoj((Lugoj));
```

Arad

Zerind

Sibiu

Timisoara

Arad

Oradea

Arad

Oradea

Fagaras

Bihor/Nicea

Arad

Lugoj

AIMA Slides ©Stuart Russell and Peter Norvig, 1998

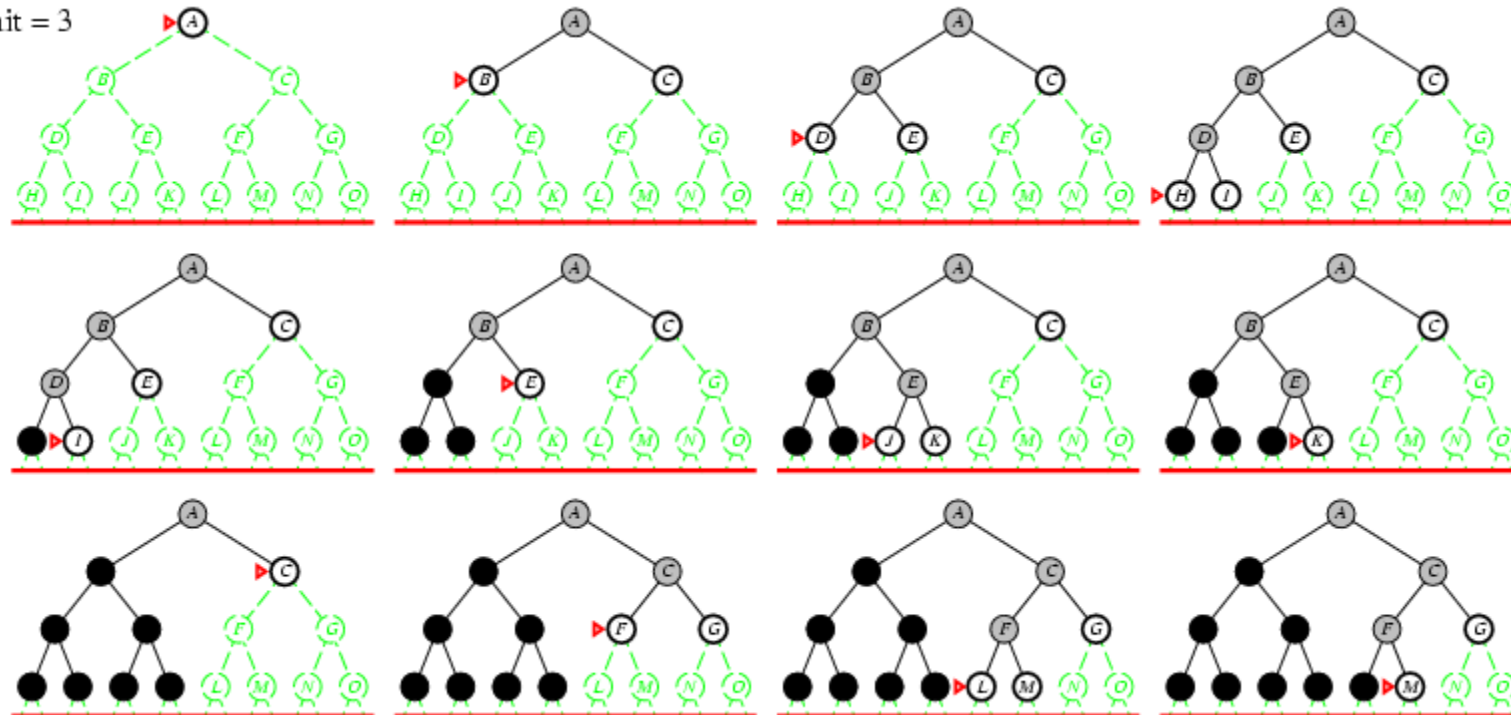
Chapter 3, Sections 1-5 60

153% 60 of 63 11 x 8.5 in

Start CS 564 laurent talks CS 561a Aima Adobe Acr... Microsoft Pow... Telnet - pollux... books Adobe Photos... 8:27 PM

Iterative deepening search

Limit = 3



Iterative deepening complexity

- In iterative deepening, nodes at bottom level are expanded once, level above twice, etc. up to root (expanded $d+1$ times) so total number of expansions is:

$$(d+1)1 + (d)b + (d-1)b^2 + \dots + 3b^{(d-2)} + 2b^{(d-1)} + 1b^d = O(b^d)$$

- In general, iterative deepening is preferred to depth-first or breadth-first when search space large and depth of solution not known.

Uniform-cost search



- For each **frontier node (queue)**, save the total cost of the path from the initial state to that node
- Expand the frontier node with the lowest path cost
- Implementation: *frontier* is a priority queue ordered by path cost
- Equivalent to breadth-first if step costs all equal
- Equivalent to Dijkstra's algorithm in general

Function **UNIFORM-COST-SEARCH**(**problem**) **returns** a solution, or failure

initialize the **explored set** to be empty

initialize the **frontier** as a **priority queue** using **node path_cost** as the **priority**

add initial state of **problem** to **frontier** with **path_cost = 0**

loop do

if the **frontier** is empty **then**

return failure

 choose a **node** and remove it from the **frontier**

if the **node** contains a goal state **then**

return the corresponding solution

 add the **node** state to the **explored set**

 for each resulting **child** from node

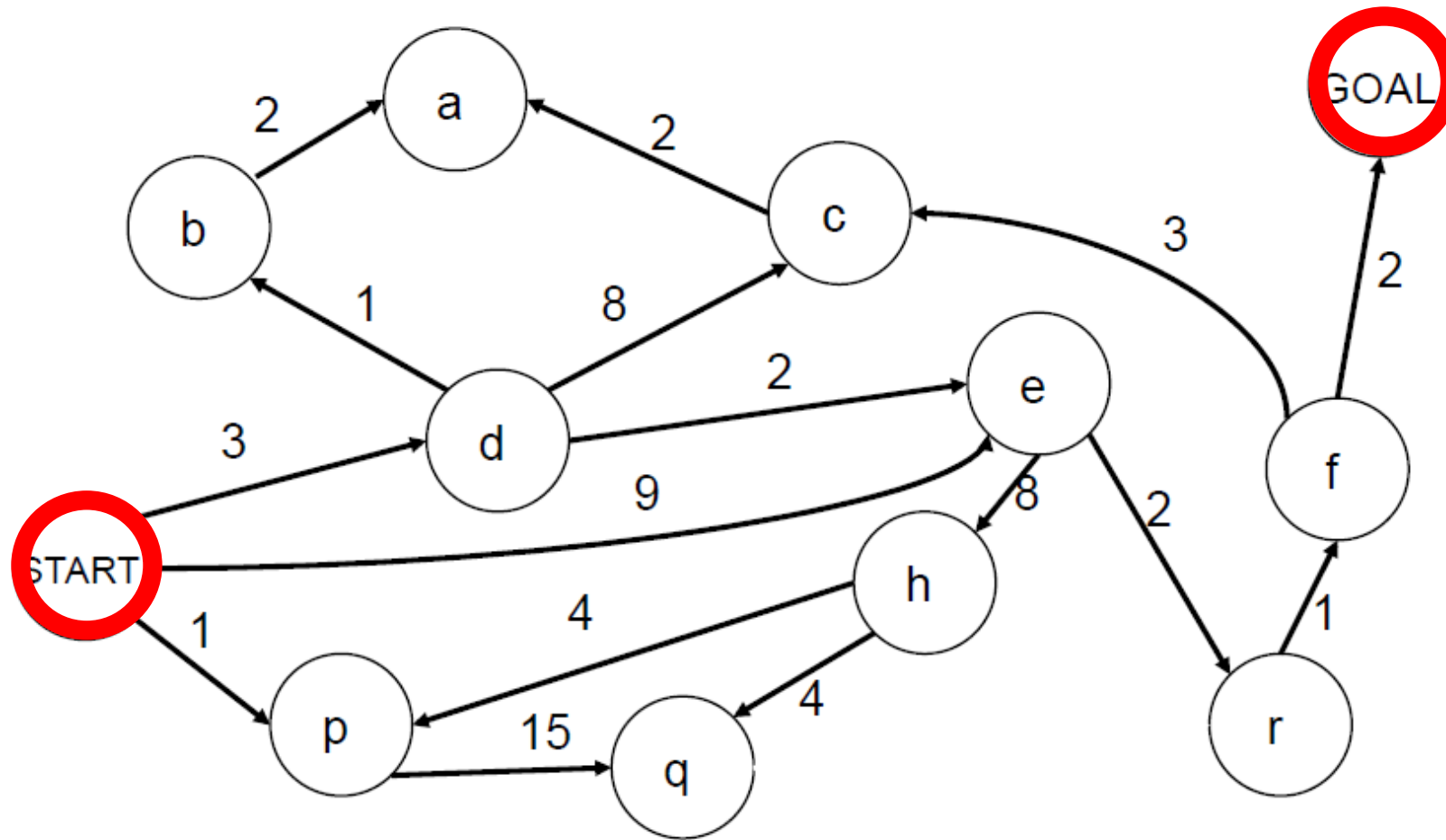
if the **child** state is not already in the **frontier** or **explored set** **then**

 add **child** to the **frontier**

else if the **child** is already in the **frontier** with higher **path_cost** **then**

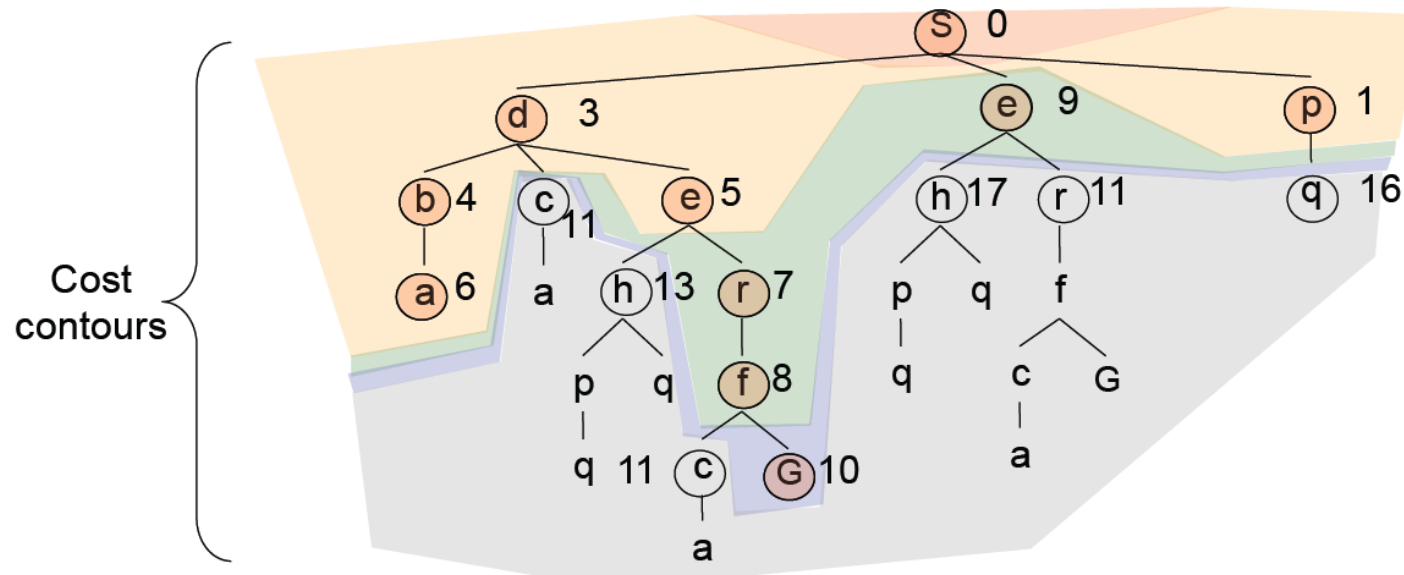
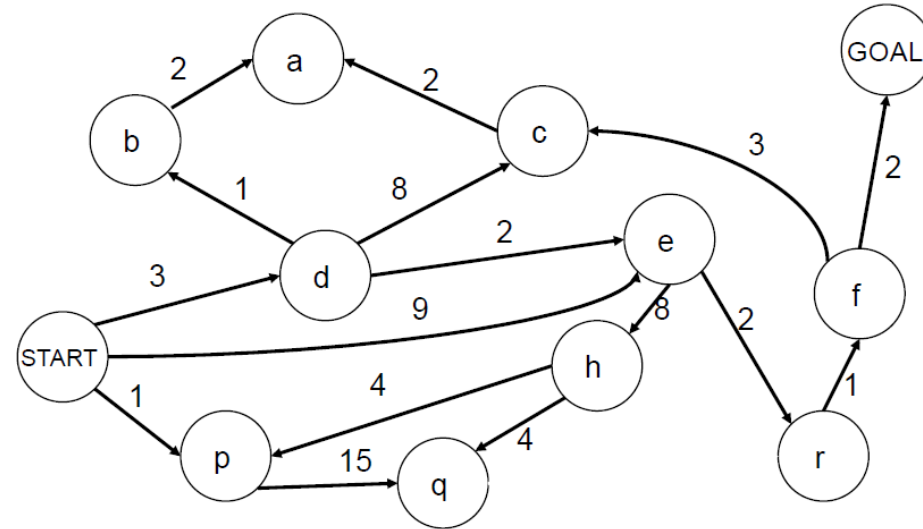
 replace that **frontier** node with **child**

Uniform-cost search example

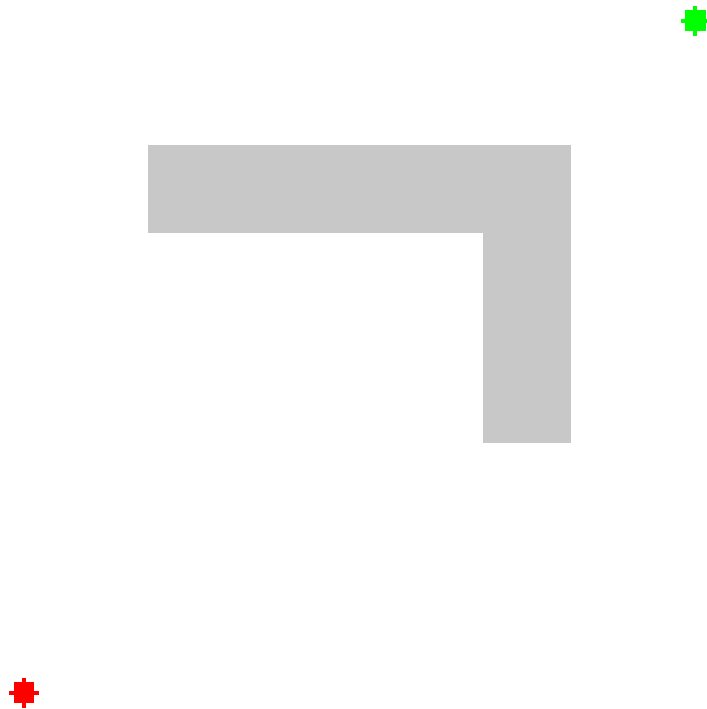


Uniform-cost search example

- Expansion order:
(S,p,d,b,e,a,r,f,e,G)



Another example of uniform-cost search



Source: [Wikipedia](#)

Properties of uniform-cost search



- **Complete?**

Yes, if step cost is greater than some positive constant ϵ (we don't want infinite sequences of steps that have a finite total cost)

- **Optimal?**

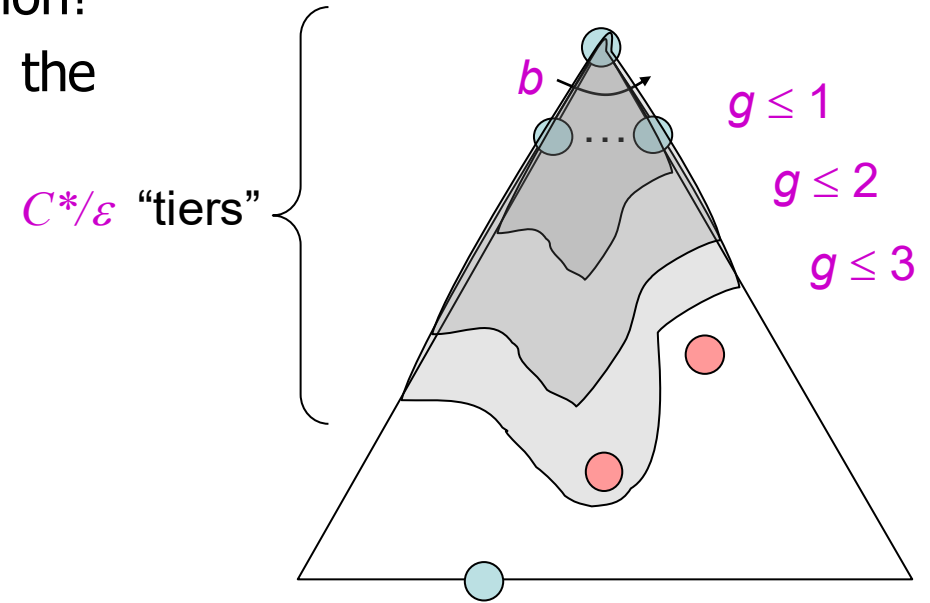
Yes!

Optimality of uniform-cost search

- **Graph separation property**: every path from the initial state to an unexplored state has to pass through a state on the frontier
 - Proved inductively
- Optimality of UCS: proof by contradiction
 - Suppose UCS terminates at goal state n with path cost $g(n)$ but there exists another goal state n' with $g(n') < g(n)$
 - By the graph separation property, there must exist a node n'' on the frontier that is on the optimal path to n'
 - But because $g(n'') \leq g(n') < g(n)$, n'' should have been expanded first!

Properties of uniform-cost search

- What nodes does UCS expand?
 - Processes all nodes with cost less than cheapest solution!
 - If that solution costs C^* and arcs cost at least ϵ , then the “effective depth” is roughly C^*/ϵ
 - Takes time $\alpha(b^{C^*/\epsilon})$ (exponential in effective depth)
- How much space does the frontier take?
 - Has roughly the last tier, so $\alpha(b^{C^*/\epsilon})$
- Is it complete?
 - Assuming C^* is finite and $\epsilon > 0$, yes!
- Is it optimal?
 - Yes!



Comparing uninformed search strategies

Criterion	Breadth first	Uniform cost	Depth-first	Depth-limited	Iterative deepening
Time	b^d	b^d	b^m	b^l	b^d
Space	b^d	b^d	bm	bl	bd
Optimal?	Yes	Yes	No	No	Yes
Complete?	Yes	Yes	No	Yes, if $l \geq d$	Yes

- b – max branching factor of the search tree
- d – depth of the least-cost solution
- m – max depth of the state-space (may be infinity)
- l – depth cutoff

Summary



- Problem formulation usually requires **abstracting away real-world details** to define a **state space** that can be explored using computer algorithms.
- Variety of uninformed search strategies; difference lies in method used to **pick node that will be further expanded**.
- Uninformed search algorithms
 - BSF (problems with memory space)
 - UCS (~BFS for finding cost-optimal solutions)
 - DFS (Tree search version has very low space requirements)
 - Depth-limited DFS (avoid infinite diving)
 - IDS (progressively increase depth limits)

References



- AIMA textbook
- Slides of CMU AI course
- Slides of UC Berkeley AI course